Start here: how to use these sheets

Four sheets: this one, then **Search**, **Fuzzy logic** and **Probability**. Every topic is laid out the same way, so you always know where to look: **What it is** (the idea in plain words), **Spot it** (words in a question that point to it), **How to do it** (steps to copy), **Formula**, **Example** (a solved one from class) and **Watch out** (common traps).

Question finder

If the question says...	Go to
probability, P(A or B), "given", without replacement, how many ways, choose, factorial, odds	**Probability 1**
prior, posterior, likelihood, "what is the probability that ... caused", reverse a conditional	**Probability 2**
state space, goal test, game tree, zero-sum, six elements, UTILITY(s, p), FSM, states and transitions	**Search 3**
Markov chain, transition matrix, hidden states, emission, Rainy / Sunny, walk / shop / clean	**Search 4**
utility score, best action, Pokemon, behavior tree, sequence, selector, decorator	**Search 5**
BFS, DFS, queue, stack, visit order, binary search tree, insert	**Search 6**
A*, f(n) = g(n) + h(n), heuristic, cheapest path, least cost	**Search 7**
minimax, MAX / MIN, back up values, alpha, beta, prune	**Search 8**
Monte Carlo, MCTS, UCB1, exploration, exploitation, Go	**Search 9**
crisp vs fuzzy, membership function, μ(x), very, extremely, somewhat (hedges)	**Fuzzy 10**
NOT / AND / OR of fuzzy sets, union, intersection, complement, De Morgan, "not very tall"	**Fuzzy 11**
IF ... THEN rules, fuzzification, rule evaluation, clipping, aggregation, centroid, defuzzify	**Fuzzy 12**
design a fuzzy system, fuzzy state machine, emotion, FLAME, affective loop	**Fuzzy 13**
classify, Naive Bayes, play golf, likelihood table, Laplace, zero probability, Gaussian	**Probability 14**
Bayesian network, DAG, CPT, joint probability from a graph, hidden node, marginalize	**Probability 15**
RBF, radial basis, kernel, spread, DTW, dynamic time warping, warping path	**Probability 16**

How to use these in the exam

1. **Read the question twice.** Underline the key words: "BFS", "membership", "given", "CPT", "utility" and so on.
2. **Find the topic** in the question finder above. It gives the sheet and the topic number.
3. **Follow "How to do it" line by line.** Write each step on your paper, even the easy ones.
4. **Compare with the Example.** If your numbers look nothing like it, check you used the right row or table.
5. **Read "Watch out"** before you move on. Most lost marks come from those traps.

Write answers that earn method marks

Part 3 of Exam 1 was marked on method. Even if your final number is wrong, you keep marks for the steps. For every calculation write:

1. The values you took from the question (for example "P(Rain) = 0.1, P(Cloud | Rain) = 0.5").
2. The formula, in letters.
3. The formula with your numbers put in.
4. The result, and one sentence saying what it means ("so a cloudy morning makes rain 12.5% likely").

What the symbols mean

`P(A)`	probability that A happens, a number from 0 (never) to 1 (always)
`P(A | B)`	probability of A when we already know B happened. Read the bar as "given".
`P(A, B) or P(A and B)`	probability that both happen
`A' or ¬A or ~A`	"not A"
`Σ`	add up all of these
`Π`	multiply all of these
`n!`	n × (n - 1) × … × 1, e.g. 4! = 24
`C(n, r)`	number of ways to choose r things from n when order does not matter
$\mu_A(x)$	"mu": how much x belongs to fuzzy set A, from 0 to 1
`∩ and ∪`	AND (intersection) and OR (union)
`α, β`	alpha and beta in alpha-beta pruning; also α = smoothing amount in Laplace
`ln, e`	natural log and e ≈ 2.718 (calculator keys "ln" and "e^x")
`∝`	"is proportional to": the same up to a constant you can ignore when comparing

Slide mistakes (use the corrected version)

- 6 children on 6 benches: 6! = **720**, not 36 (the teacher corrected it).
- Conditional rule: P(B | A) = P(A and B) ÷ P(A). The slide shows "×". Divide.
- Math given science: P(M | S) = 0.40 / 0.60 = **0.67** (teacher corrected).
- Complement: P(A') = 1 - P(A).
- Laplace example: (3 + 1) / (100 + 2) = **4/102**, not 3/102.
- Spam word "money": 80 / 1000 = **0.08**, not 0.8.
- Alarm network: P(no burglary) = **0.999**; 0.998 is P(no fire).
- Alpha-beta slide p56 shades too much: only the last pair of leaves (-1, 0) is really pruned.

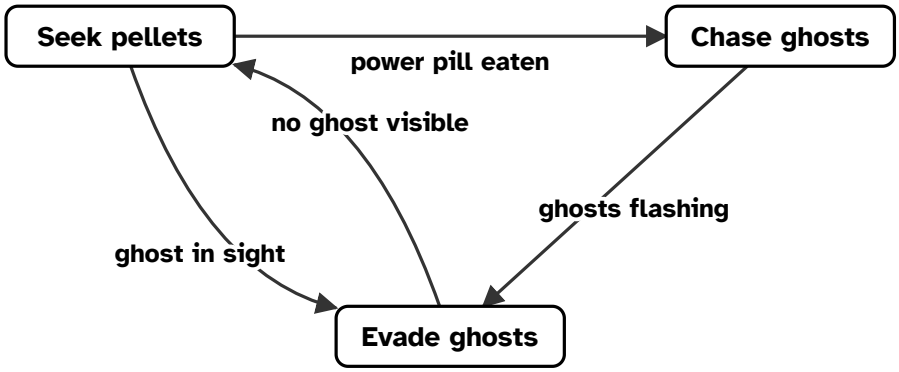
Search (deck 6, taught 15 Sep)

3 Problem solving, games and state machines

WHAT IT IS

The words used to describe a problem for a computer. A **search problem** has a **state space** (every possible situation), a **start state**, a **goal test** (a check: is this the goal?) and a **solution**, which is the list of actions from start to goal (the **plan**). A **game** adds an opponent. A **finite state machine (FSM)** is a diagram of states (circles) joined by transitions (arrows) that happen when an event occurs.

PICTURE



3 states (boxes) and 4 transitions (arrows). The words on each arrow are the event that makes the change happen.

SPOT IT

"Define the problem", "name the six elements", "draw the FSM", "what is the node / transition / action", zero-sum, perfect information.

HOW TO DO IT

1. Search problem: list state space, start state, actions, goal test, and say the solution is the path of actions.
2. Game, six elements: S_0 (start position), TO-MOVE(s) (whose turn), ACTIONS(s) (legal moves), RESULT(s, a) (the position after a move), IS-TERMINAL(s) (is the game over?), UTILITY(s, p) (final score for player p, only at the end).
3. FSM: draw one circle per state, one arrow per change, and write the event on each arrow.

FORMULA

Studied games are: deterministic (no dice), 2 players, take turns, perfect information (everyone sees everything), zero-sum (my gain = your loss). MAX moves first and wants a high score; MIN wants it low.

EXAMPLE

Ms Pac-Man FSM: 3 states (Seek pellets, Chase ghosts, Evade ghosts). Seek → Chase when a power pill is eaten; Chase → Evade when ghosts flash; Seek → Evade when a ghost is in sight; Evade → Seek when no ghost is visible. Chess utility: win 1, loss 0, draw 0.5.

WATCH OUT

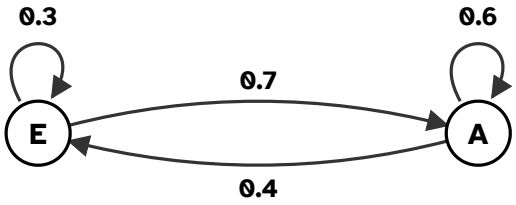
- Nodes are states, arrows are transitions. Do not swap them.
- Utility exists only at the end of the game (terminal states).
- Perfect information is about seeing everything; deterministic is about no luck. Different things.

4 Markov chains and hidden Markov models

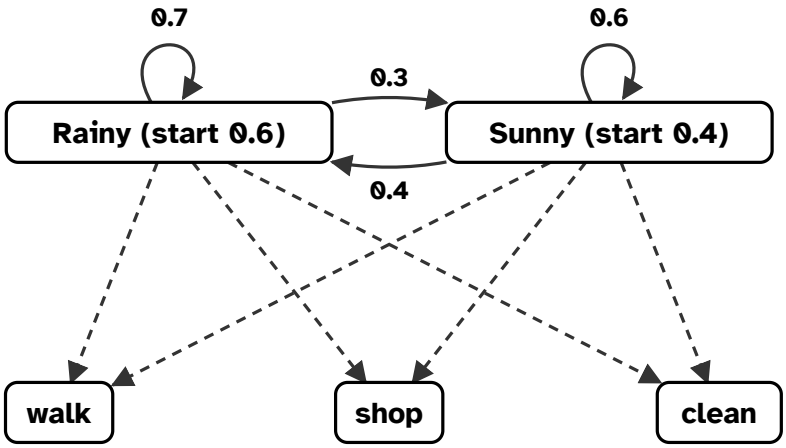
WHAT IT IS

A **Markov chain** is an FSM where each arrow has a probability. The next state depends **only on the current state**. In a **hidden Markov model (HMM)** you cannot see the state (the weather); you only see something it causes (what a person does).

PICTURE



Markov chain: arrows leaving a state add up to 1.



HMM: you see the bottom row, not the weather. Dashed lines are emissions.

Emission	walk	shop	clean
from Rainy	0.1	0.4	0.5
from Sunny	0.6	0.3	0.1

SPOT IT

transition matrix, "probability of moving", hidden states, observations, emission, Rainy / Sunny with walk / shop / clean.

HOW TO DO IT

1. Name the five HMM parts: hidden states, observations, start probabilities, transition probabilities (state to state), emission probabilities (state to observation).
2. Probability of one hidden path with what you saw: multiply start × emission × transition × emission × … along the path.
3. Two steps in a chain: add up every route that ends where you want, multiplying along each route.
4. Most likely next state (teacher's way): pick the larger probability on the arrows leaving the current state.

FORMULA

E/A chain: E→E 0.3, E→A 0.7, A→E 0.4, A→A 0.6 (each row adds to 1).
Rainy/Sunny: start R 0.6, S 0.4. R→R 0.7, R→S 0.3, S→R 0.4, S→S 0.6. Rainy emits walk 0.1, shop 0.4, clean 0.5. Sunny emits walk 0.6, shop 0.3, clean 0.1.

EXAMPLE

Hidden Rainy then Rainy, seen clean then shop: 0.6 × 0.5 × 0.7 × 0.4 = **0.084**.
From E, in A after two steps: 0.3 × 0.7 + 0.7 × 0.6 = **0.63**.

WATCH OUT

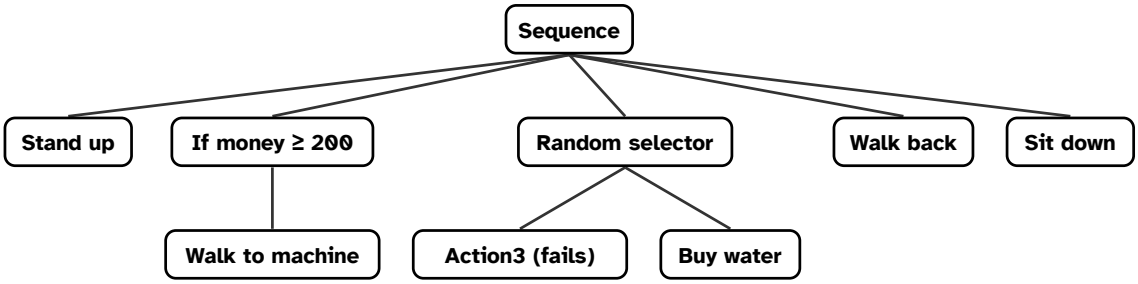
- Emission = state to observation. Transition = state to state.
- Each row of a transition table adds up to 1.
- For two steps, add all routes, not just the obvious one.

5 Utility-based AI and behavior trees

WHAT IT IS

Utility is a score for how good an action is right now (0 to 1, or 0 to 10). The AI scores every action and does the one with the highest score. A **behavior tree** is a tree that runs an NPC's actions; every node answers success or failure.

PICTURE



Sequence: all children must succeed, left to right. Selector: one success is enough. "If money ≥ 200" is a decorator.

Pokemon move	Score	Reason
Tackle	7	enemy is weak; normal move does full damage to fire
Growl	1	no damage
Leech Seed	0	enemy already seeded, no effect
Vine Whip	5	grass on fire does half damage

SPOT IT

"best action", "score each option", Pokemon, utility curve, behavior tree, sequence, selector, decorator.

HOW TO DO IT

1. Utility: list the facts of the situation, give each action a score with a one-line reason, pick the highest.
2. Behavior tree: **sequence** runs children left to right and fails at the first failure (all must succeed). **Selector** tries children until one succeeds. **Decorator** adds a condition ("if money ≥ 200") or a loop ("until health = 0").
3. Trace a tree: walk it left to right and write success or failure at each node.

EXAMPLE

Pokemon vs a weak, seeded fire type: Tackle 7, Growl 1, Leech Seed 0 (already seeded), Vine Whip 5 (grass on fire does half). Choose **Tackle**.
Ms Pac-Man: low threat, hunt ghosts; high threat, evade ghosts.

WATCH OUT

- Sequence needs ALL children to succeed; selector needs ONE.
- Utility (how good) is not probability (how likely).

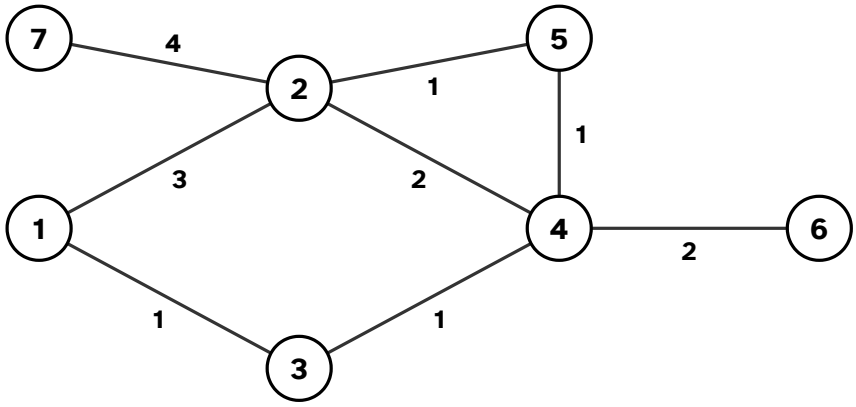
6 Uninformed search: BFS, DFS and binary search trees

WHAT IT IS

Ways to explore a graph with no hint about where the goal is. **BFS** (breadth-first) goes level by level using a **queue** (first in, first out). **DFS** (depth-first) goes as deep as it can, then backs up, using a **stack** (last in, first out).

PICTURE

	BFS (breadth-first)	DFS (depth-first)
Order	level by level	as deep as possible, then back up
Uses	a queue: first in, first out	a stack: last in, first out
Finds	the path with the fewest edges	any path, no guarantee
Looks at costs?	no	no



Assignment 5: start 1, goal 7. BFS visits 1, 2, 3, 4, 5, 7; DFS visits 1, 2, 4, 3, 5, 6, 7; both find 1-2-7. See worked problems P9 and P10.

SPOT IT

BFS, DFS, visit order, queue, stack, "which path is found", Assignment 5, binary search tree, insert the numbers.

HOW TO DO IT

1. Write each node's neighbours in a fixed order (alphabetical or smallest number first). Say which order you use.
2. BFS: start the queue with the start node. Take the front node out, write it in the visit order, add its unvisited neighbours to the back. Repeat until the goal is taken out.
3. DFS: from the current node go to its first unvisited neighbour; when stuck, go back to the last node that still has an unvisited neighbour. Stop at the goal.
4. Path: follow each node's parent (the node that found it) back from the goal.
5. BST insert: start at the root; smaller goes left, bigger goes right; repeat at each node until an empty spot.

FORMULA

BFS finds the path with the **fewest edges** (on unweighted graphs). DFS gives no guarantee. Neither looks at edge costs.

EXAMPLE

Workshop graph (A→B 1, A→C 4, B→C 2, B→D 3, C→E 5, D→F 2, D→G 4, E→G 3, F→G 1): BFS visits A, B, C, D, E, F, G, path A-B-D-G (cost 8). DFS visits A, B, C, E, G, path A-B-C-E-G (cost 11).
Assignment 5 (start 1, goal 7): BFS 1, 2, 3, 4, 5, 7; DFS 1, 2, 4, 3, 5, 6, 7; both reach 7 by 1-2-7.
BST 5, 7, 10, 14, 1, 3, 8, 4, 2: root 5; 5 has left 1, right 7; 7 right 10; 10 left 8, right 14; 1 right 3; 3 left 2, right 4.

WATCH OUT

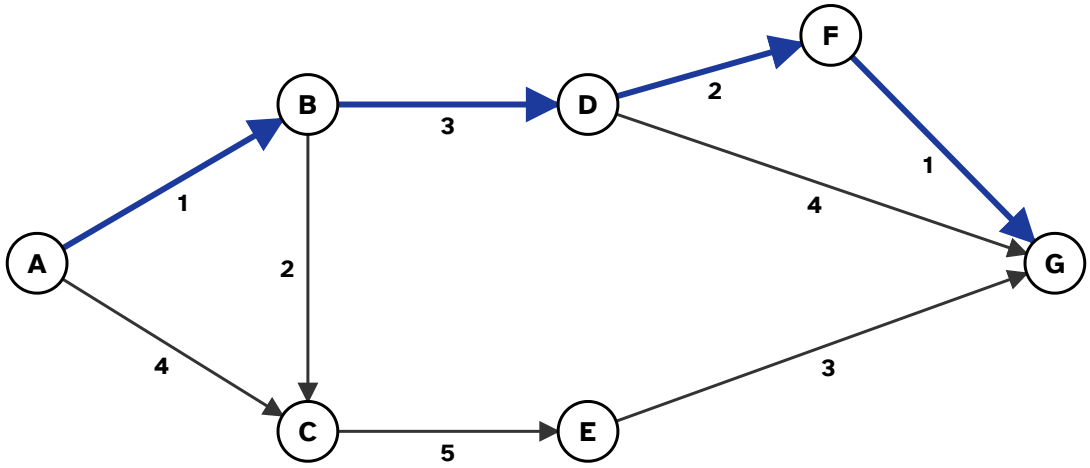
- BFS = queue, DFS = stack. Easy to swap.
- BFS's path is not always the cheapest when edges have costs.
- In a BST compare at every level, not only with the root.

7 Informed search: A*

WHAT IT IS

A search that uses a guess of the distance left. It always opens the node with the smallest **f(n) = g(n) + h(n)**: cost so far plus estimated cost to the goal. It finds the cheapest path.

PICTURE



Class workshop: A-B-D-F-G costs 1 + 3 + 2 + 1 = 7, the cheapest. BFS would return A-B-D-G (fewest edges, cost 8).

SPOT IT

A*, f(n), g(n), h(n), heuristic, cheapest path, least cost, Assignment 5 part 3.

HOW TO DO IT

1. Put the start in a list with g = 0 and f = h(start).
2. Take out the node with the smallest f. If it is the goal, stop.
3. For each neighbour: g = g of this node + edge cost; f = g + h. If the neighbour already has a bigger g, replace it.
4. Repeat. Write the list with f values at every step.
5. No h values given? Do what the teacher did: list every path from start to goal, add up its costs, pick the smallest. (Same as A* with h = 0.)

FORMULA

```
f(n) = g(n) + h(n)
```

A good h never overestimates the real distance left ("admissible").

EXAMPLE

Workshop graph, all paths A to G: A-B-D-F-G = 1 + 3 + 2 + 1 = **7**; A-B-D-G = 8; A-B-C-E-G = 11; A-C-E-G = 12. Answer A-B-D-F-G, cost 7.

Assignment 5: path 1-2-7, cost 3 + 4 = 7.

WATCH OUT

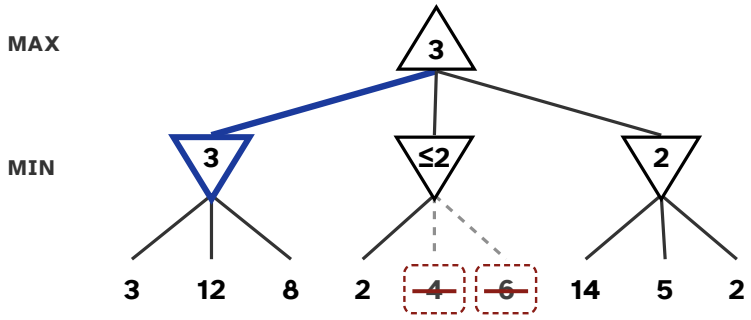
- Stop when the goal is TAKEN OUT of the list, not when you first see it.
- Use f = g + h, not h alone and not the smallest single edge.
- Update a node when you find a cheaper way to it.

8 Adversarial search: minimax and alpha-beta

WHAT IT IS

How to choose a move against an opponent. Leaves at the bottom of the game tree hold scores for MAX. Going up, **MAX** levels take the biggest child and **MIN** levels take the smallest. **Alpha-beta pruning** skips branches that cannot change the answer.

PICTURE



Triangle up = MAX (takes the largest), triangle down = MIN (takes the smallest). After MIN1 = 3, the second MIN node sees 2 ≤ 3, so 4 and 6 are skipped (struck out).

SPOT IT

minimax, MAX / MIN levels, "value of the root", "which move", alpha, beta, prune, "which leaves are not evaluated".

HOW TO DO IT

1. Minimax: start at the bottom. At each MAX node write the largest child value; at each MIN node the smallest. The root value tells MAX which move to take.
2. Alpha-beta: α = best MAX has so far (starts at -∞), β = best MIN has so far (starts at +∞). Go left to right.
3. At a MAX node, after each child: α = max(α, value). At a MIN node: β = min(β, value).
4. As soon as β ≤ α, stop: the remaining children of that node are pruned. Cross them out.

FORMULA

```
prune when β ≤ α
```

Pruning never changes the root value. Checking good moves first prunes more.

EXAMPLE

Root MAX with MIN children (3, 12, 8), (2, 4, 6), (14, 5, 2): MIN values 3, 2, 2, root **3**. With alpha-beta, after the first child α = 3; the second child's first leaf 2 makes β = 2 ≤ 3, so 4 and 6 are pruned.

WATCH OUT

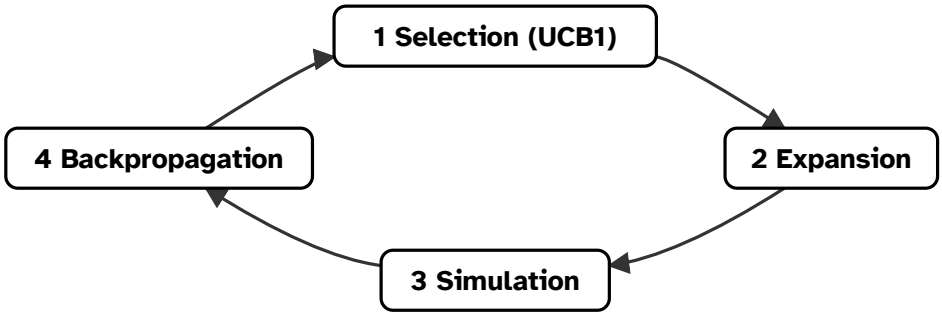
- MIN takes the SMALLEST value; leaf scores are always from MAX's point of view.
- Prune only after a child has been checked and β ≤ α.
- With three players, each player picks the child best for its own number in the score vector.

9 Monte Carlo tree search (MCTS)

WHAT IT IS

For huge games like Go (about 10^{170} positions, about 300 moves per turn), MCTS plays many random games from the current position and keeps statistics on which moves win. It builds the tree only where it looks promising.

PICTURE



Repeat until time runs out, then play the most visited move.

SPOT IT

Monte Carlo, MCTS, Go, AlphaGo, UCB1, exploration vs exploitation, four steps.

HOW TO DO IT

1. **Selection:** from the root, keep choosing the child with the highest UCB1.
2. **Expansion:** add one new child node.
3. **Simulation** (rollout): play random moves to the end of the game; win = +1, loss = -1 or 0.
4. **Backpropagation:** add 1 visit and the result to every node on the path back to the root.
5. Repeat until time runs out, then play the most visited move.

FORMULA

$$UCB1 = v_i + C \times \sqrt{(\ln N / n_i)}$$

v_i = the child's win rate (exploitation: use what works). The square-root part (exploration: try the unknown) is big for children with few visits. N = parent visits, n_i = child visits. A child with 0 visits is tried first.

EXAMPLE

N = 20, C = √2: child A 6 wins / 10 visits: 0.6 + 1.414 × √(2.996 / 10) ≈ 1.374. Child B 3 / 5: 0.6 + 1.414 × √(2.996 / 5) ≈ 1.695. Pick **B**: same win rate, fewer visits.

WATCH OUT

- Order: selection, expansion, simulation, backpropagation.
- MCTS needs no heuristic. A* does.
- Which to use: A* for finding a route, minimax for small board games, MCTS for huge games.

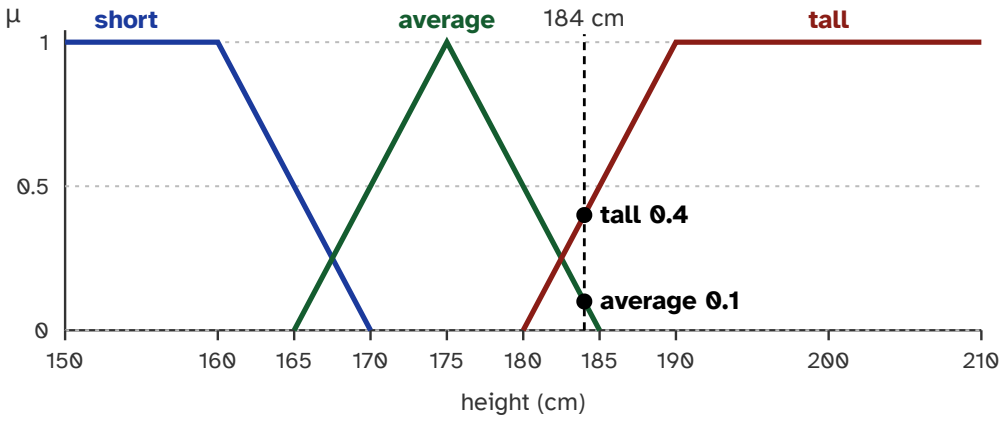
Fuzzy logic (deck 7, taught 22 Sep)

10 Fuzzy sets, membership functions and hedges

WHAT IT IS

Normal (crisp) sets say yes or no: 9.9 m is "seen", 10.1 m is not. A **fuzzy set** lets something belong **partly**, with a degree from 0 to 1 called its **membership**, written μ(x). A 184 cm man is "tall" to 0.4. **Fuzzy is not probability:** fuzzy is how true a word is for something that already exists; probability is the chance something will happen.

PICTURE



Read a membership by going up from the height to each line. 184 cm is tall to 0.4 and average to 0.1 at the same time.

Hedge	Rule	Tom (tall 0.86)
a little	$\mu^{1.3}$	0.822
slightly	$\mu^{1.7}$	0.774
very	μ^{2}	0.7396
extremely	μ^{3}	0.6361
very very	μ^{4}	0.547
somewhat, more or less	$\sqrt{\mu}$	0.927
indeed	$2\mu^2$ if $\mu \le 0.5$, else $1 - 2(1 - \mu)^2$	0.9608

SPOT IT

crisp vs fuzzy, membership function, μ(x), degree, linguistic variable, very, extremely, somewhat, Tom 0.86.

HOW TO DO IT

1. Membership functions come first. Draw each word (short, average, tall) as a shape over the number line: triangle, trapezoid or S-curve.
2. To read a membership on a slope: (distance from where the slope starts) ÷ (length of the slope).
3. Hedges change a membership: very = square it; extremely = cube it; very very = power 4; somewhat / more or less = square root.
4. Notation: tall = (0/180, 0.5/185, 1/190) means membership 0 at 180 cm, 0.5 at 185 cm, 1 at 190 cm. The "/" is not division.

FORMULA

Heights: short = 1 up to 160, down to 0 at 170. Average = triangle 0 at 165, 1 at 175, 0 at 185. Tall = 0 up to 180, up to 1 at 190.

Hedges: a little $\mu^{1.3}$, slightly $\mu^{1.7}$, very μ^{2}, extremely μ^{3}, very very μ^{4}, somewhat $\sqrt{\mu}$, indeed $2\mu^2$ (if $\mu \le 0.5$) or $1 - 2(1 - \mu)^2$.

EXAMPLE

184 cm: tall = (184 - 180) / 10 = **0.4**; average = (185 - 184) / 10 = **0.1**.

Tom tall 0.86: very tall 0.86^2 = **0.7396**; extremely tall 0.86^3 = **0.6361**.

WATCH OUT

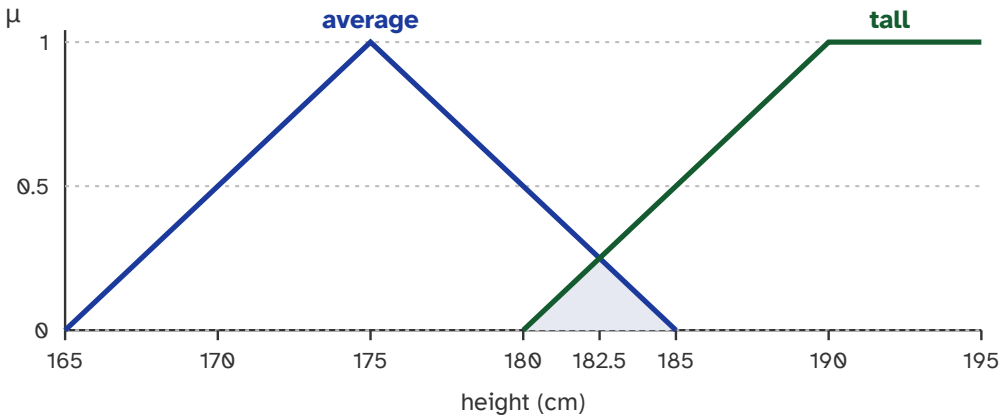
- Very = square, not double (2 × 0.86 would be more than 1).
- Memberships in different sets do not have to add up to 1.
- Do not call a membership a probability.

11 Fuzzy set operations

WHAT IT IS

How to combine fuzzy sets. Work **point by point**: at each x, combine the two memberships.

PICTURE



AND (intersection) = the lower line at each height: the shaded part, highest at 182.5 cm with 0.25. OR (union) = the upper line.

Operation	Word	Rule	Example (μ_A 0.7, μ_B 0.4)
complement	NOT A	$1 - \mu_A$	0.3
intersection	A AND B	min	0.4
union	A OR B	max	0.7

SPOT IT

NOT, AND, OR, complement, intersection, union, contained, De Morgan, "not very tall and not very short".

HOW TO DO IT

1. NOT A: $1 - \mu_A$.
2. A AND B (intersection): the smaller of the two, min.
3. A OR B (union): the larger of the two, max.
4. For a phrase, work from the inside: hedge first, then NOT, then AND / OR.

FORMULA

$$\text{NOT: } 1 - \mu \qquad \text{AND: } \min(\mu_A, \mu_B) \qquad \text{OR: } \max(\mu_A, \mu_B)$$

A is inside B if $\mu_A(x) \leq \mu_B(x)$ everywhere (very tall is inside tall). De Morgan: NOT(A AND B) = NOT A OR NOT B.

EXAMPLE

"Not very tall and not very short" (class exercise): $\mu_C = \min(1 - \mu_{tall}^2, 1 - \mu_{short}^2)$. For tall 0.6, short 0.1: min(1 - 0.36, 1 - 0.01) = min(0.64, 0.99) = **0.64**. "Very very" uses power 4 instead of 2.

WATCH OUT

- AND = min, OR = max (not product, not sum).
- "Not very tall" = $1 - \mu^2$, not $(1 - \mu)^2$.

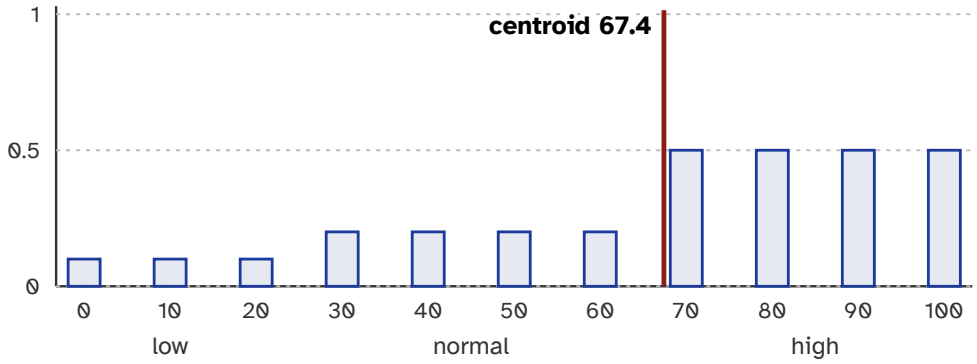
12 Fuzzy rules and Mamdani inference (four steps)

WHAT IT IS

How a fuzzy system turns input numbers into one output number using rules like "IF funding is inadequate THEN risk is high". The IF part is the **antecedent**, the THEN part is the **consequent**.

PICTURE

Rule	Inputs	Operator	Strength	Output set clipped
1	A3 0, B1 0.1	OR = max	0.1	low
2	A2 0.2, B2 0.7	AND = min	0.2	normal
3	A1 0.5	one input	0.5	high



After clipping, the three outputs are put together (aggregation). The centroid (red line) is the balance point: 209 ÷ 3.1 = 67.4.

SPOT IT

IF … THEN, fuzzification, rule evaluation, clipping, scaling, aggregation, defuzzification, centroid, project risk, 67.4.

HOW TO DO IT

1. **Fuzzification:** read each input's membership in every set from the graphs.
2. **Rule evaluation:** for each rule, combine its inputs (AND = min, OR = max) to get one number, the rule strength. Write it for every rule. The teacher calls this the "selection process".
3. Cut (**clip**) each rule's output shape flat at its rule strength. (Scaling shrinks the whole shape instead; the teacher always uses clipping.)
4. **Aggregation:** put all the clipped shapes together into one shape.
5. **Defuzzification:** find the balance point of that shape (the centroid).

FORMULA

$$\text{centroid} = \Sigma\ (\ \mu(z) \times z\ ) \div \Sigma\ \mu(z)$$

Take sample points z (every 10), multiply each by its height, add them up, then divide by the sum of the heights.

EXAMPLE

Project risk: memberships A1 0.5, A2 0.2, A3 0; B1 0.1, B2 0.7. Rule 1: max(A3, B1) = max(0, 0.1) = 0.1 → low. Rule 2: min(A2, B2) = min(0.2, 0.7) = 0.2 → normal. Rule 3: A1 = 0.5 → high. Heights 0.1 at z = 0 to 20, 0.2 at 30 to 60, 0.5 at 70 to 100. Centroid = (30 × 0.1 + 180 × 0.2 + 340 × 0.5) ÷ (3 × 0.1 + 4 × 0.2 + 4 × 0.5) = 209 ÷ 3.1 = **67.4**.

WATCH OUT

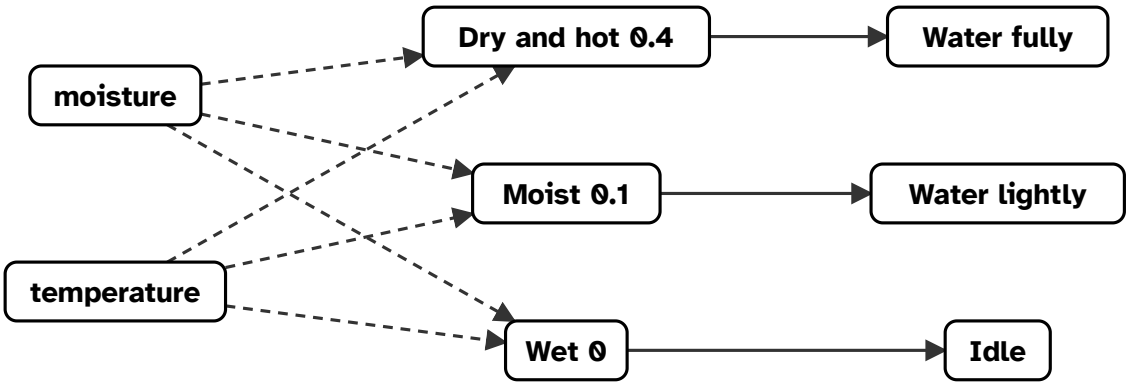
- Do not stop at the biggest rule. Clip, combine all, then compute the centroid.
- Divide by the sum of the heights (3.1), not by the number of points.

13 Fuzzy logic in game AI, and the big workshop

WHAT IT IS

Fuzzy inputs feed utility scores, so an NPC can act gradually instead of switching suddenly. A **fuzzy state machine** can be in several states at once (Patrol 30%, Alert 70%). **FLAME** (Fuzzy Logic Adaptive Model of Emotions) turns events into emotions with strengths, like anger 0.85, that fade over time.

PICTURE



Draw it like an HMM: inputs on the left, hidden states with degrees in the middle (dashed = hidden part), actions on the right.

SPOT IT

"design a fuzzy system", fuzzy state machine, fuzzy transitions, emotion, FLAME, affective loop, the big workshop.

HOW TO DO IT

1. Pick a scenario with 2 inputs and 1 output (e.g. watering: soil moisture and temperature → valve opening).
2. Name the fuzzy set and its subsets (dry, moist, wet; cool, hot; low, medium, high).
3. Give each subset a membership shape with numbers, and sketch the graphs.
4. Write 3 rules. Pick input values and do the four steps of topic 12 with numbers.
5. Say how it is utility AI: each clipped rule output scores an action; combining them gives the decision.
6. Draw the fuzzy state machine like an HMM: inputs → hidden states with degrees → output actions.

EXAMPLE

Watering, moisture 32, temperature 32: dry 0.4, moist 0.1, hot 0.7. Rule "dry AND hot → high" = min(0.4, 0.7) = 0.4; "moist OR cool → medium" = 0.1. Centroid (180 × 0.1 + 340 × 0.4) ÷ (0.4 + 1.6) = **77**: open the valve 77%.
Other facts: affective loop = elicitation, sensing, detection, adaptation. Layers: FSM (structure), fuzzy (nuance), utility (brain), emotion (soul).

WATCH OUT

- The teacher took marks off for: skipping the min / max for each rule, picking one rule instead of aggregating, and an FSM that does not start from the inputs.

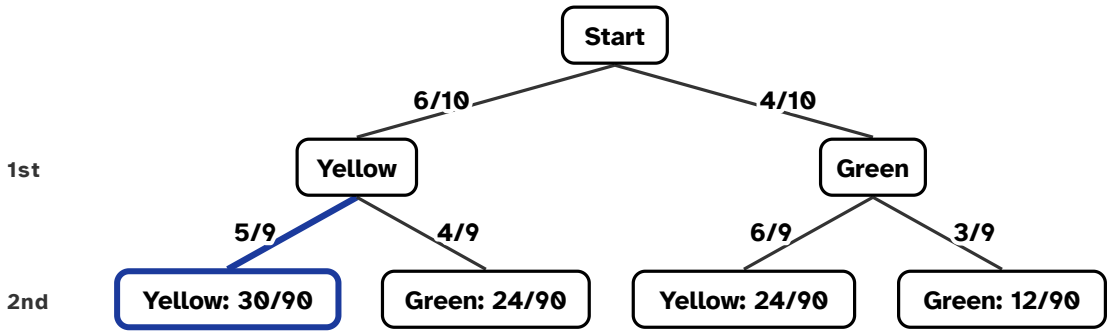
Probability (deck 8, taught 6 Oct)

1 Probability basics and counting

WHAT IT IS

Probability is a number from 0 (impossible) to 1 (certain). Counting tells you how many ways something can happen, so you can turn it into a probability.

PICTURE



Jar of 4 green and 6 yellow. Multiply along a branch for "and"; add different branches for "or". The second-draw fractions change because the first marble is not put back.

SPOT IT

"probability that", "or", "and", "given", without replacement, how many ways, arrange, choose, odds.

HOW TO DO IT

1. Equally likely outcomes: P = good outcomes ÷ all outcomes.
2. "A or B": add, then subtract the overlap once.
3. "A and B" one after the other: multiply. Without replacement, the second fraction uses what is left.
4. "Given": divide the "both" probability by the probability of what you were given.
5. Arranging n different things in order: n!. Choosing r from n where order does not matter: C(n, r).

FORMULA

```
P(A or B) = P(A) + P(B) - P(A and B)    P(A') = 1 - P(A)
```

```
P(B | A) = P(A and B) ÷ P(A)    P(A and B) = P(A) × P(B | A)
```

```
C(n, r) = n! ÷ ( r! × (n - r)! )
```

Course odds = losing : winning. Odds 99 to 1 means P = 1 / 100.

EXAMPLE

Jar of 4 green, 6 yellow, draw 2 without putting back, both yellow: 6/10 × 5/9 = **1/3**.
Choose 4 from 10: C(10, 4) = **210**. Five cards from 52: **2,598,960**.
Powerball: 1 ÷ (C(69, 5) × 26) = 1 / **292,201,338**.

WATCH OUT

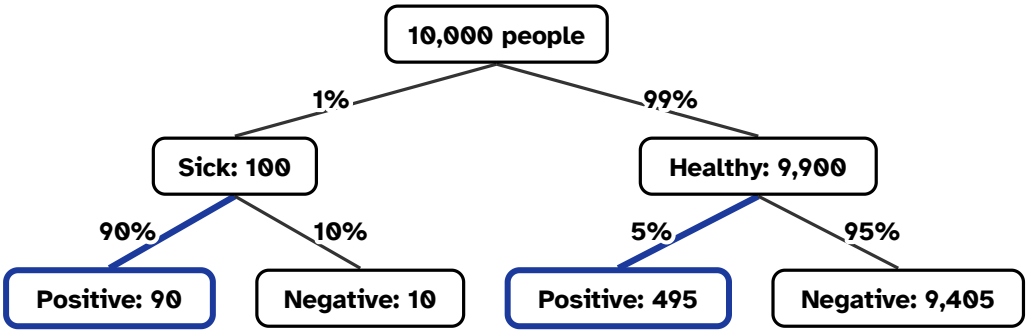
- 6 children on 6 benches = 6! = 720 (not 36).
- Without replacement the second draw changes (5/9, not 6/10).
- Independent (one does not affect the other) is not the same as mutually exclusive (cannot both happen).

2 Bayes' theorem

WHAT IT IS

A way to turn around a conditional probability. You often know "how likely the evidence is if the cause is true" and want "how likely the cause is now that I saw the evidence".

PICTURE



Bayes' theorem in counts: of the 90 + 495 = 585 positive tests, only 90 are sick. P(sick | positive) = 90 ÷ 585 ≈ 0.154.

SPOT IT

prior, posterior, likelihood, evidence, "given that we observed", "what is the chance it was caused by", picnic, flu and fever, test results.

HOW TO DO IT

1. Name the cause H and the evidence X.
2. Write down P(X | H) (likelihood), P(H) (prior: before the evidence) and P(X) (evidence).
3. If P(X) is not given, build it: add P(X | cause) × P(cause) over every possible cause.
4. Put the numbers into the formula.

FORMULA

$$P(H \mid X) = P(X \mid H) \times P(H) \div P(X)$$

$$P(X) = \Sigma\ P(X \mid \text{cause}) \times P(\text{cause})$$

Posterior = likelihood × prior ÷ evidence.

EXAMPLE

Picnic: P(Cloud | Rain) = 0.5, P(Rain) = 0.1, P(Cloud) = 0.4. P(Rain | Cloud) = 0.5 × 0.1 ÷ 0.4 = **0.125** (12.5%).
Flu: 0.9 × 0.05 ÷ 0.09 = **0.5**.

WATCH OUT

- P(Rain | Cloud) is not P(Cloud | Rain). Reversing them is the whole point.
- Do not forget the prior.

14 Naive Bayes classifier and Laplace smoothing

WHAT IT IS

A way to sort a new example into a class (play golf: yes or no) using a table of past examples. For each class, multiply how common the class is by how often each feature value appeared in that class. The bigger score wins. "Naive" because it treats the features as unrelated.

PICTURE

Feature	Value	Yes (of 9)	No (of 5)
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Temp	Hot	2	2
	Mild	4	2
	Cool	3	1
Humidity	High	3	4
	Normal	6	1
Windy	False	6	2
	True	3	3

Play golf counts. Likelihood = count ÷ class size, for example P(Sunny | Yes) = 3/9. Priors: P(Yes) = 9/14, P(No) = 5/14.

SPOT IT

classify, play golf, "will the customer buy", likelihood table, frequency table, Laplace, zero probability, Gaussian.

HOW TO DO IT

1. Count each class: prior = class count ÷ total rows.
2. For each feature value in the new example, count rows in that class with that value, divide by the class count.
3. Score for each class = prior × all those fractions multiplied.
4. The biggest score is the answer. For percentages, divide each score by the sum of the scores.
5. If any fraction is 0, use Laplace: add 1 to the count and add the number of possible values to the bottom.

FORMULA

$$\text{score}(C) = P(C) \times P(x_1 \mid C) \times P(x_2 \mid C) \times \ldots$$

$$\text{Laplace: } (\text{count} + \alpha) \div (N + \alpha K), \text{ usually } \alpha = 1$$

$$\text{Gaussian (numbers): } 1 \div (\sqrt{2\pi}\,\sigma) \times e^{-(x-\mu)^2 / (2\sigma^2)}$$

EXAMPLE

Golf: P(Yes) = 9/14, P(No) = 5/14. New day (Rainy, Cool, High, Windy): Yes = 2/9 × 3/9 × 3/9 × 3/9 × 9/14 = 0.00529; No = 3/5 × 1/5 × 4/5 × 3/5 × 5/14 = 0.02057. **No, do not play** (0.2 vs 0.8).

Golf tables (Yes of 9, No of 5): Sunny 3, 2; Overcast 4, 0; Rainy 2, 3; Hot 2, 2; Mild 4, 2; Cool 3, 1; High 3, 4; Normal 6, 1; Not windy 6, 2; Windy 3, 3.

Spam: "money" 80 times in 1000 words: 0.08; with Laplace (vocabulary 1000): 81 ÷ 2000 = 0.0405.

WATCH OUT

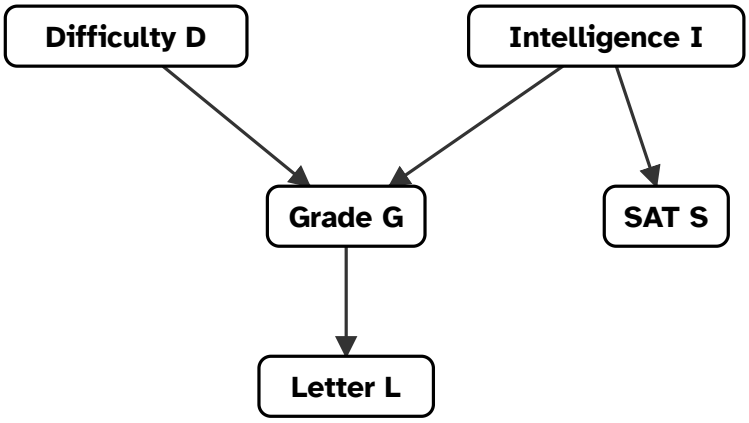
- Divide by the class count (9), not the total (14).
- Multiply by the prior at the end.
- One 0 makes the whole product 0; that is why Laplace exists.

15 Bayesian networks

WHAT IT IS

A diagram of causes and effects with no loops (a DAG). Each circle is a variable; an arrow means "affects". Each circle has a table (CPT) of its probabilities given its parents. Multiply one number from each table to get the probability of a whole situation.

PICTURE



No arrow in: P(D), P(I). Arrows in: P(G | I, D), P(S | I), P(L | G).

Example: P(i1, d0, g2, s1, l0)	
P(i1)	0.3
P(d0)	0.6
P(g2 | i1, d0)	0.08
P(s1 | i1)	0.8
P(l0 | g2)	0.4
multiply	0.004608

SPOT IT

Bayesian network, DAG, CPT, "find P(...)" from a graph, student network, sprinkler, burglary alarm, hidden node, marginalize.

HOW TO DO IT

1. Teacher's method: look at the arrow heads. A circle with no arrow coming in: write P(it). A circle with arrows coming in: write P(it | its parents).
2. Write the whole statement first, then look up each number in its table.
3. Multiply.
4. If a variable is not mentioned in the question (hidden), do the calculation for each of its values and add them.
5. For P(A | B): compute P(A, B) and P(B) (summing hidden variables), then divide.

FORMULA

$$P(X_1, \ldots, X_n) = \Pi\ P(X_i \mid \text{parents of } X_i)$$

$$P(x) = \Sigma_y\ P(x, y) \quad \text{(sum out what you do not need)}$$

EXAMPLE

Student: P(i1, d0, g2, s1, l0) = P(i1) P(d0) P(g2 | i1, d0) P(s1 | i1) P(l0 | g2) = 0.3 × 0.6 × 0.08 × 0.8 × 0.4 = **0.004608**.
Sprinkler: P(Rain, no sprinkler, grass not wet) = 0.2 × 0.99 × 0.2 = **0.0396**.
Alarm: 0.95 × 0.80 × 0.001 × 0.999 × 0.998 ≈ **0.00076**.
Hidden H: P(+w | +s, -g) = 0.0875 ÷ 0.1583 = **0.553**; P(-w | -s, +g) = **0.866**.

WATCH OUT

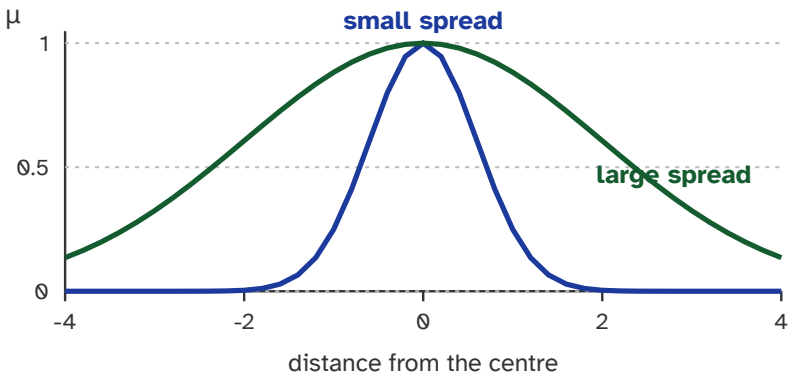
- Use the CPT row that matches ALL the parents' values (grass wet depends on sprinkler AND rain).
- A loop (A→B→C→A) is not allowed.
- P(no burglary) = 0.999; 0.998 is P(no fire).

16 RBF networks and dynamic time warping

WHAT IT IS

RBF (radial basis function): a bell-shaped score that is 1 at a centre and falls as you move away. An RBF network wraps each class in a round region; a normal neural network (MLP) uses straight lines. **DTW** (dynamic time warping) compares two sequences that may be stretched in time, like the same word said slowly and quickly.

PICTURE



Small spread: narrow, picky. Large spread: wide, accepts points far from the centre.

X \ Y	1	2	3	4
1	**0**	**1**	3	6
3	2	1	**1**	2
4	5	3	2	**1**

DTW table D for (1, 3, 4) and (1, 2, 3, 4). Marked cells are the path; the bottom-right cell is the distance.

SPOT IT

RBF, kernel, spread, radbas, DTW, warping path, cost matrix, Euclidean vs DTW.

HOW TO DO IT

1. RBF: put the distance into the formula. A small spread (σ) gives a narrow, picky bell; a large spread a wide one.
2. DTW step 1: make a table C of $|a_i - b_j|$ for every pair.
3. DTW step 2: make table D. First row and first column are running totals. Every other cell = its C value + the smallest of the cells above, left and diagonally up-left.
4. The bottom-right cell is the DTW distance.
5. Step 3: walk back from the bottom-right, always to the smallest neighbour. That path is the warping path (diagonal = match).

FORMULA

$$\text{RBF: } k(x, y) = e^{-||x - y||^2 / (2\sigma^2)} \qquad \text{radbas}(n) = e^{-n^2}$$

$$D(i, j) = C(i, j) + \min(D(i-1, j), D(i, j-1), D(i-1, j-1))$$

Euclidean: rigid 1-to-1, cost O(n). DTW: elastic, cost O(n × m).

EXAMPLE

DTW of (1, 3, 4) and (1, 2, 3, 4): C rows (0 1 2 3), (2 1 0 1), (3 2 1 0). D rows (0 1 3 6), (2 1 1 2), (5 3 2 1). Distance = **1**.

WATCH OUT

- Add C(i, j) to the minimum; do not just copy the minimum.
- A bigger spread is LESS picky, not more.

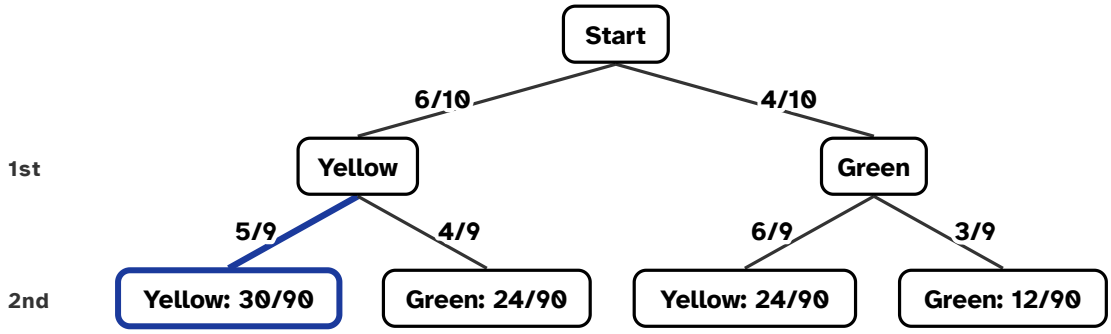
Worked problems: every type, step by step

One solved problem for every kind of question. Every step is written out. To answer your own question, copy the same steps with your numbers. The topic number on each problem points to the sheet that explains the idea.

P1 Probability without replacement

TOPIC 1

Problem. A jar has 4 green and 6 yellow marbles. You take 2 marbles, one after the other, without putting the first one back. (a) What is the probability that both are yellow? (b) What is the probability that at least one is green?



Each branch shows its probability. Multiply along a path. The four ends add up to 90/90 = 1.

SOLUTION, STEP BY STEP

1. Count the marbles: 4 + 6 = 10.
2. First marble yellow: 6 yellow out of 10, so 6/10.
3. One yellow marble is now gone. There are 9 marbles left and 5 of them are yellow. Second marble yellow: 5/9. (This is why "without replacement" matters: the second fraction changes.)
4. Both things must happen, so multiply along that path of the tree: 6/10 × 5/9 = 30/90.
5. Simplify: 30/90 = 1/3 ≈ 0.333. That is (a).
6. "At least one green" happens every time "both yellow" does not. So use the complement rule: 1 - 1/3 = 2/3 ≈ 0.667. That is (b). Check with the tree: 24/90 + 24/90 + 12/90 = 60/90 = 2/3.

Answer. (a) 1/3 ≈ 0.333. (b) 2/3 ≈ 0.667.

P2 Counting: arrangements and combinations

TOPIC 1

Problem. (a) In how many ways can 6 children sit on 6 benches, one child per bench? (b) A lottery picks 6 numbers from 1 to 45 and the order does not matter. What is the probability that one ticket wins?

SOLUTION, STEP BY STEP

1. First decide: does the order matter? In (a) it does (who sits on which bench). In (b) it does not (a ticket is just a set of numbers).
2. (a) Every child is used once and order matters, so use a factorial. Bench 1 has 6 children to choose from, bench 2 has the 5 left, then 4, 3, 2, 1.
3. Multiply: 6 × 5 × 4 × 3 × 2 × 1 = 720. (The slide's 6 × 6 = 36 is wrong; the teacher corrected it.)
4. (b) Order does not matter, so use a combination: C(45, 6) = 45! ÷ (6! × 39!).
5. Cancel 39! from the top and bottom, which leaves the 6 biggest numbers on top: (45 × 44 × 43 × 42 × 41 × 40) ÷ (6 × 5 × 4 × 3 × 2 × 1).
6. Top: 45 × 44 × 43 × 42 × 41 × 40 = 5,864,443,200. Bottom: 6! = 720.
7. Divide: 5,864,443,200 ÷ 720 = 8,145,060 possible tickets.
8. Only one ticket wins, and every ticket is equally likely, so the probability is 1 ÷ 8,145,060.

Answer. (a) 720 ways. (b) 1 / 8,145,060.

P3 Bayes' theorem with a total probability

TOPIC 2

Problem. 1% of people have a disease. A test is positive for 90% of sick people and for 5% of healthy people. A person tests positive. What is the probability that the person is sick?

SOLUTION, STEP BY STEP

1. Name things: H = the person is sick (the cause). X = the test is positive (the evidence). We want P(H | X).
2. Write down what the question gives:

Quantity	Meaning	Value
P(H)	prior: sick before the test	0.01
P(not H)	healthy	1 - 0.01 = 0.99
P(X | H)	positive if sick	0.9
P(X | not H)	positive if healthy	0.05

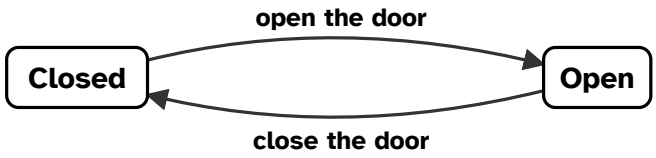
3. P(X) is not given, so build it from both cases (total probability): P(X) = P(X | H) × P(H) + P(X | not H) × P(not H).
4. Put in the numbers: 0.9 × 0.01 + 0.05 × 0.99 = 0.009 + 0.0495 = 0.0585.
5. Bayes' theorem: P(H | X) = P(X | H) × P(H) ÷ P(X) = 0.009 ÷ 0.0585.
6. Divide: 0.009 ÷ 0.0585 ≈ 0.154.
7. Read it in words: of every 10,000 people, 90 are sick and test positive, but 495 healthy people also test positive. Only 90 of the 585 positives are sick.

Answer. About 0.154 (15.4%). Most positive tests come from healthy people, because there are so many more healthy people.

P4 Describe a game and draw an FSM

TOPIC 3

Problem. (a) Write tic-tac-toe as a game using the six elements. (b) Draw the finite state machine of a door.



The answer to (b): boxes are states, arrows are transitions, and the words on an arrow are the event that causes it.

SOLUTION, STEP BY STEP

1. (a) Write the six elements one by one:

Element	Tic-tac-toe
S_0, the start state	the empty 3 × 3 board
TO-MOVE(s), whose turn	X if X and O have the same number of marks, otherwise O. X is MAX.
ACTIONS(s), legal moves	the empty squares
RESULT(s, a), the next state	the same board with the current player's mark on square a
IS-TERMINAL(s), is it over?	true if a row, column or diagonal has three of the same mark, or the board is full
UTILITY(s, X), the final score	+1 if X wins, -1 if O wins, 0 for a draw

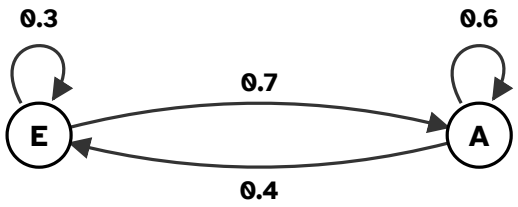
2. Check: utility is only defined when the game is over (terminal states).
3. (b) Draw one box per state: Closed and Open.
4. Draw one arrow per change of state, and write the event on it: Closed to Open is "open the door"; Open to Closed is "close the door". See the picture.

Answer. The table in step 1, and the door FSM in the picture (2 states, 2 transitions).

P5 Markov chain: probability after two steps

TOPIC 4

Problem. A Markov chain has two states, E and A, shown below. You start in E. What is the probability of being in A after two steps?



The probabilities on the arrows leaving a state add up to 1: from E, 0.3 + 0.7; from A, 0.4 + 0.6.

SOLUTION, STEP BY STEP

1. List every route of exactly two steps that starts in E and ends in A:

Route	Step 1	Step 2	Multiply
E → E → A	stay in E: 0.3	E to A: 0.7	0.3 × 0.7 = 0.21
E → A → A	E to A: 0.7	stay in A: 0.6	0.7 × 0.6 = 0.42

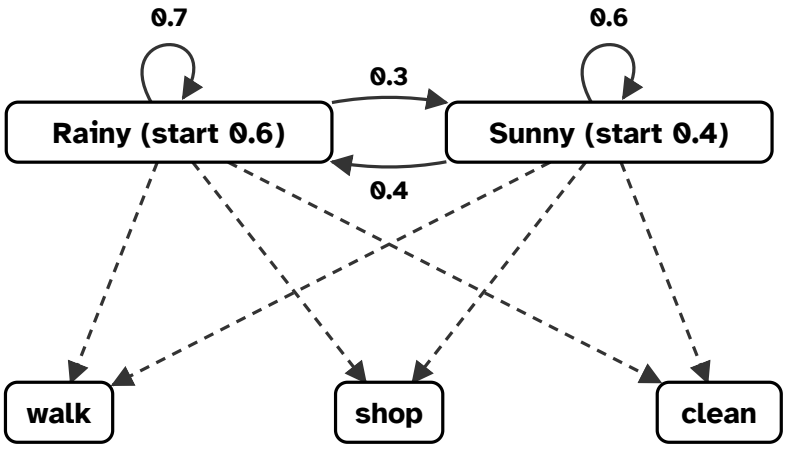
2. Within a route, both steps must happen, so multiply.
3. The two routes cannot both happen, so add them: 0.21 + 0.42 = 0.63.
4. Check: the probability of being in E after two steps is 1 - 0.63 = 0.37.

Answer. 0.63.

P6 Hidden Markov model: probability of one hidden path

TOPIC 4

Problem. Use the hidden Markov model below. What is the probability that day 1 is Rainy and day 2 is Sunny, and you see clean on day 1 and walk on day 2?



Solid arrows: transitions between the hidden states, with their probabilities. Dashed lines: emissions (numbers in the table).

Emission	walk	shop	clean
from Rainy	0.1	0.4	0.5
from Sunny	0.6	0.3	0.1

SOLUTION, STEP BY STEP

1. Write the path in time order: start in Rainy, Rainy shows clean, move from Rainy to Sunny, Sunny shows walk.
2. Pick each number from the picture:

Piece	Value
P(start Rainy)	0.6
P(clean | Rainy), emission	0.5
P(Rainy to Sunny), transition	0.3
P(walk | Sunny), emission	0.6

3. Everything must happen, so multiply in order. First: 0.6 × 0.5 = 0.3.
4. Next: 0.3 × 0.3 = 0.09.
5. Last: 0.09 × 0.6 = 0.054.

Answer. 0.054.

P7 Choose an action with utility scores

TOPIC 5

Problem. A guard scores two actions from its health h and ammo a (both from 0 to 100): U(attack) = (h ÷ 100) × (a ÷ 100) and U(flee) = 1 - h ÷ 100. The guard has h = 30 and a = 80. What does it do?

SOLUTION, STEP BY STEP

1. Turn the inputs into fractions: h ÷ 100 = 30 ÷ 100 = 0.3, and a ÷ 100 = 80 ÷ 100 = 0.8.
2. Score each action:

Action	Formula	Score
Attack	0.3 × 0.8	0.24
Flee	1 - 0.3	0.7

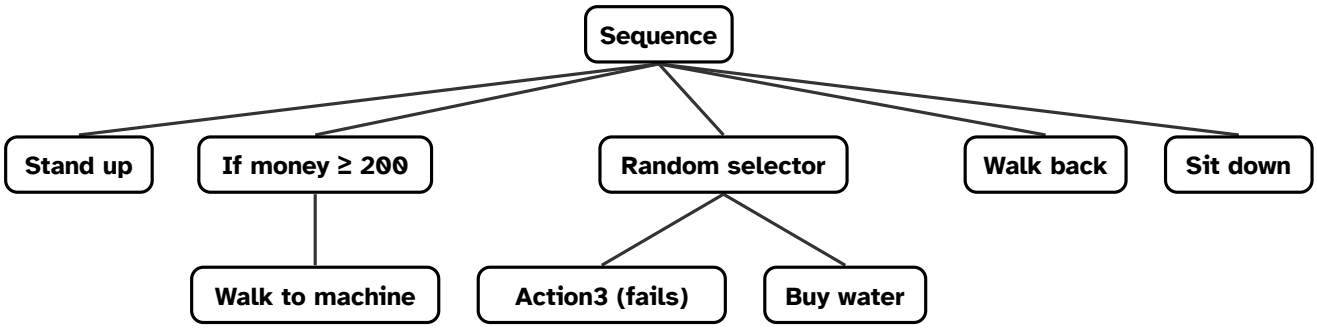
3. Compare the scores: 0.7 is bigger than 0.24.
4. Utility AI always does the action with the highest score.

Answer. Flee (utility 0.7 against 0.24).

P8 Trace a behavior tree

TOPIC 5

Problem. Trace the behavior tree below once with 250 baht and once with 150 baht. Say what the root returns each time.



A sequence runs its children left to right and stops at the first failure. A selector tries children until one succeeds. "If money ≥ 200" is a decorator (a condition).

SOLUTION, STEP BY STEP

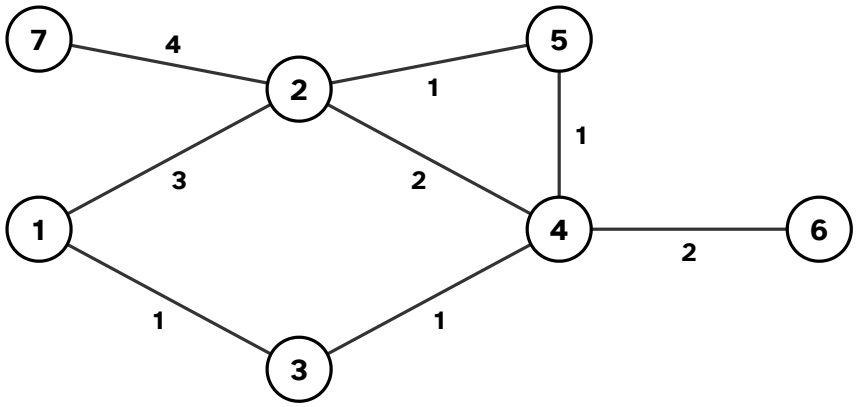
1. 250 baht, child 1: stand up. Success.
2. Child 2: the decorator checks 250 ≥ 200. True, so it runs "walk to machine". Success.
3. Child 3: the selector tries Action3: fails. It tries "buy water": success. One child succeeded, so the selector succeeds.
4. Children 4 and 5: walk back, sit down. Both succeed. Every child succeeded, so the sequence returns success.
5. 150 baht, child 1: stand up. Success.
6. Child 2: the decorator checks 150 ≥ 200. False, so it returns failure without running "walk to machine".
7. A sequence stops at the first failure. Children 3, 4 and 5 never run. The root returns failure.

Answer. 250 baht: success (buys water and sits down). 150 baht: failure (stands up, then stops).

P9 BFS: visit order and path

TOPIC 6

Problem. Run breadth-first search on the graph below from node 1 to node 7. When a node has several neighbours, take the smallest number first. Give the visit order and the path found.



Start at 1. Goal 7. Numbers on the lines are costs.

SOLUTION, STEP BY STEP

1. Write each node's neighbours in order (smallest first): 1: 2, 3. 2: 1, 4, 5, 7. 3: 1, 4. 4: 2, 3, 5, 6. 5: 2, 4. 6: 4. 7: 2.
2. BFS uses a queue: take nodes out from the front, add new ones at the back. Record each new node's parent (the node that found it).
3. Work through it:

Take out	New neighbours added (parent)	Queue after
(start)	1	[1]
1	2, 3 (parent 1)	[2, 3]
2	4, 5, 7 (parent 2)	[3, 4, 5, 7]
3	none (1 and 4 already seen)	[4, 5, 7]
4	6 (parent 4)	[5, 7, 6]
5	none	[7, 6]
7	goal: stop	

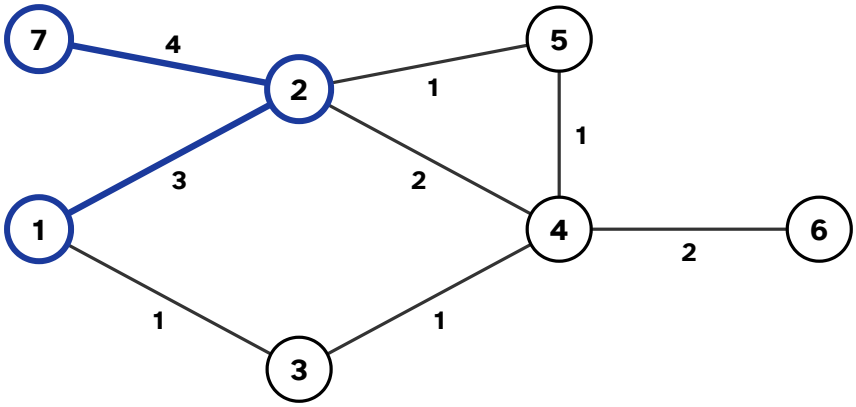
4. Visit order is the order nodes were taken out: 1, 2, 3, 4, 5, 7. (6 was added but never taken out.)
5. Path: start at the goal and follow the parents back: 7 came from 2, and 2 came from 1. Reverse it: 1-2-7.

Answer. Visit order 1, 2, 3, 4, 5, 7. Path 1-2-7 (cost 3 + 4 = 7).

P10 DFS: visit order and path

TOPIC 6

Problem. Run depth-first search on the same graph from node 1 to node 7, smallest neighbour first. Give the visit order and the path found.



The path 1-2-7 is drawn thick.

SOLUTION, STEP BY STEP

1. DFS goes as deep as it can: from the current node, go to its first unvisited neighbour. When there is none, go back one node and try again.
2. Work through it:

At	Unvisited neighbours	Move
1	2, 3	go to 2
2	4, 5, 7	go to 4
4	3, 5, 6	go to 3
3	none	back to 4
4	5, 6	go to 5
5	none	back to 4
4	6	go to 6
6	none	back to 4, then back to 2
2	7	go to 7: goal, stop

3. Visit order is the order nodes were first reached: 1, 2, 4, 3, 5, 6, 7.
4. Path: 7 was reached from 2, and 2 from 1, so the path is 1-2-7 (drawn thick).

Answer. Visit order 1, 2, 4, 3, 5, 6, 7. Path 1-2-7.

P11 Build a binary search tree

TOPIC 6

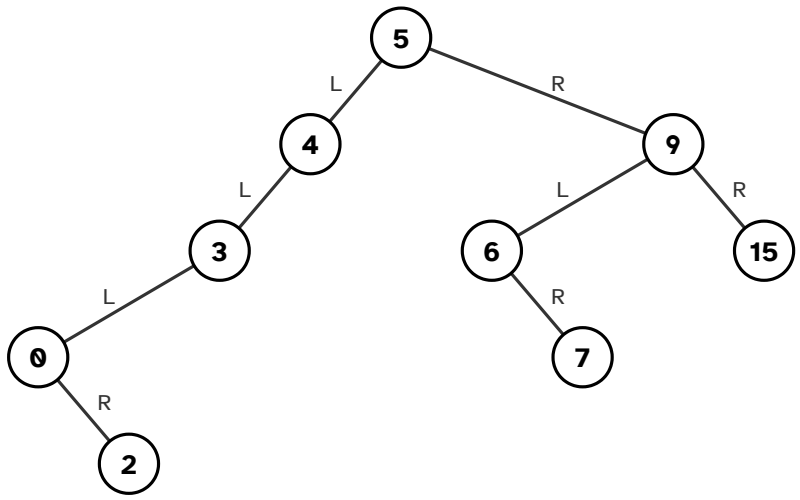
Problem. Insert 5, 9, 4, 3, 0, 15, 6, 7, 2 in that order into an empty binary search tree. Draw the tree.

SOLUTION, STEP BY STEP

1. Rule: a smaller value goes left, a bigger value goes right. Compare at every node on the way down, not only at the root.
2. Insert one value at a time:

Value	Path down the tree	Placed
5	empty tree	root
9	9 > 5	right of 5
4	4 < 5	left of 5
3	3 < 5, 3 < 4	left of 4
0	0 < 5, 0 < 4, 0 < 3	left of 3
15	15 > 5, 15 > 9	right of 9
6	6 > 5, 6 < 9	left of 9
7	7 > 5, 7 < 9, 7 > 6	right of 6
2	2 < 5, 2 < 4, 2 < 3, 2 > 0	right of 0

3. Draw the result:



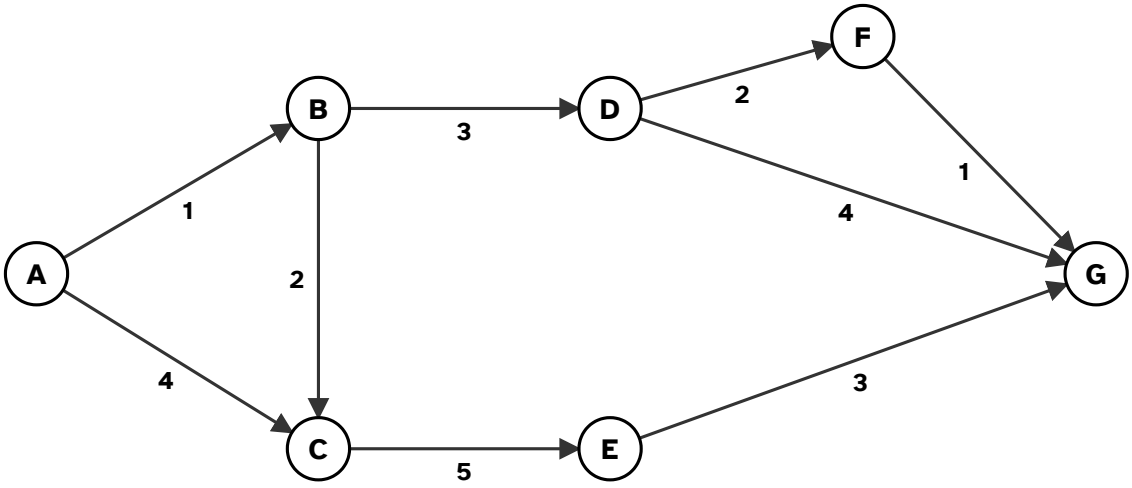
L and R mark left and right children. Reading the tree left to right gives the values in order: 0, 2, 3, 4, 5, 6, 7, 9, 15.

Answer. Root 5. 5: left 4, right 9. 9: left 6, right 15. 4: left 3. 3: left 0. 0: right 2. 6: right 7.

P12 A* when no heuristic is given

TOPIC 7

Problem. Find the cheapest path from A to G in the graph below. No heuristic values are given.



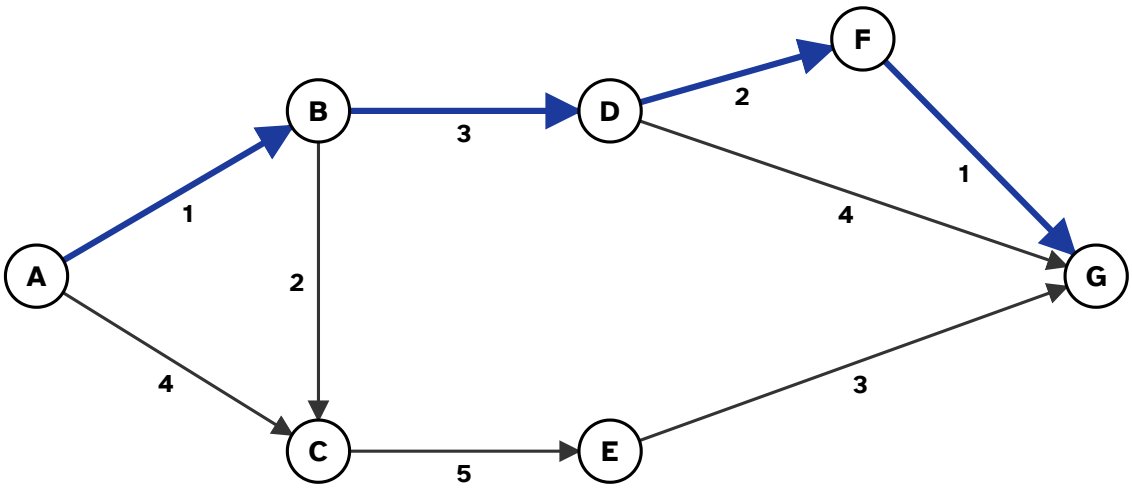
Start A, goal G. Arrows show the direction you may travel; numbers are costs.

SOLUTION, STEP BY STEP

1. No h values are given, so do what the teacher did in class: list every path from A to G and add up its costs. (This is the same as A* with h = 0.)
2. Follow the arrows to find every path, then add:

Path	Costs	Total
A-B-D-F-G	1 + 3 + 2 + 1	7
A-B-D-G	1 + 3 + 4	8
A-B-C-E-G	1 + 2 + 5 + 3	11
A-C-E-G	4 + 5 + 3	12

3. Pick the smallest total: 7.
4. The answer path:



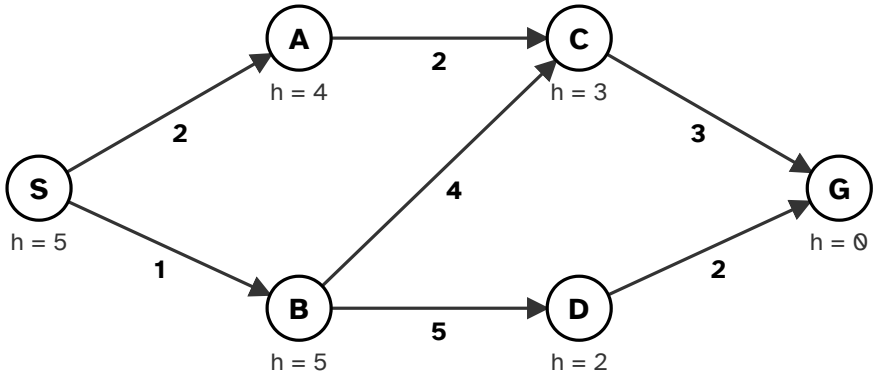
The cheapest path A-B-D-F-G is drawn thick.

Answer. A-B-D-F-G with cost 7.

P13 A* with a heuristic

TOPIC 7

Problem. Use A* to find the path from S to G in the graph below. The number under each node is its heuristic h. When two nodes have the same f, take them in alphabetical order.



SOLUTION, STEP BY STEP

1. Rule: always take out the node with the smallest f = g + h, where g is the cost from S so far. Stop when G is taken out.
2. Keep a table of the open list after each step:

Take out	Work	Open list after (f)
(start)	S: g = 0, f = 0 + 5 = 5	S 5
S	A: g = 2, f = 2 + 4 = 6. B: g = 1, f = 1 + 5 = 6	A 6, B 6
A (tie, A before B)	C: g = 2 + 2 = 4, f = 4 + 3 = 7	B 6, C 7
B	C through B: g = 1 + 4 = 5, worse than 4, keep old C. D: g = 1 + 5 = 6, f = 6 + 2 = 8	C 7, D 8
C	G: g = 4 + 3 = 7, f = 7 + 0 = 7	G 7, D 8
G	goal taken out: stop	

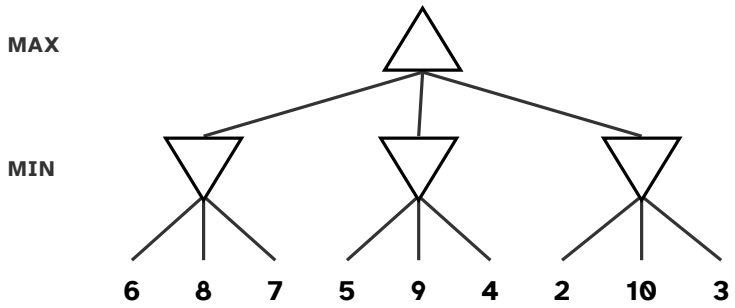
3. Read the path back through the parents: G came from C, C from A, A from S. Path S-A-C-G.
4. Add the costs to check: 2 + 2 + 3 = 7. D was never taken out, so the heuristic saved work.

Answer. S-A-C-G with cost 7.

P14 Minimax and alpha-beta pruning

TOPIC 8

Problem. The game tree below has MAX at the root (triangle pointing up) and three MIN nodes (triangles pointing down). (a) Find the minimax value. (b) Which leaves does alpha-beta skip, going left to right?

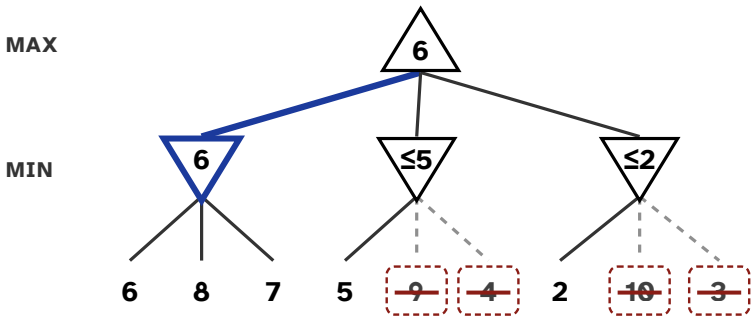


SOLUTION, STEP BY STEP

1. (a) Work from the bottom. Each MIN node takes its smallest leaf: MIN1 = min(6, 8, 7) = 6. MIN2 = min(5, 9, 4) = 4. MIN3 = min(2, 10, 3) = 2.
2. The root is MAX, so it takes the largest: max(6, 4, 2) = 6. MAX should move to MIN1.
3. (b) Alpha-beta. Start with α = -∞ (best MAX has found) and β = +∞ (best MIN has found). Prune as soon as β ≤ α.
4. Work left to right:

Where	Leaf seen	α	β	What happens
MIN1	6	-∞	6	keep going
MIN1	8	-∞	6	keep going
MIN1	7	-∞	6	MIN1 = 6; root α becomes 6
MIN2	5	6	5	β 5 ≤ α 6: prune 9 and 4
MIN3	2	6	2	β 2 ≤ α 6: prune 10 and 3

5. The solved tree (crossed leaves were never looked at):



6. The root value is still 6: pruning never changes the answer. 5 of the 9 leaves were looked at.

Answer. (a) 6, so MAX moves to MIN1. (b) Skipped leaves: 9, 4, 10, 3.

P15 MCTS: choose a child with UCB1

TOPIC 9

Problem. In Monte Carlo tree search, the root has been visited N = 20 times. Child A has 6 wins in 10 visits, child B 3 wins in 5 visits, child C 1 win in 5 visits. Use UCB1 = win rate + C × √(ln N ÷ n) with C = √2 ≈ 1.414. Which child is selected?

SOLUTION, STEP BY STEP

1. Find ln N with the calculator: ln 20 ≈ 2.996.
2. Fill in a table, one row per child:

Child	Win rate	ln N ÷ n	√	× 1.414 (bonus)	UCB1
A	6 ÷ 10 = 0.6	2.996 ÷ 10 = 0.2996	0.547	0.774	0.6 + 0.774 ≈ 1.374
B	3 ÷ 5 = 0.6	2.996 ÷ 5 = 0.599	0.774	1.095	0.6 + 1.095 ≈ 1.695
C	1 ÷ 5 = 0.2	2.996 ÷ 5 = 0.599	0.774	1.095	0.2 + 1.095 ≈ 1.295

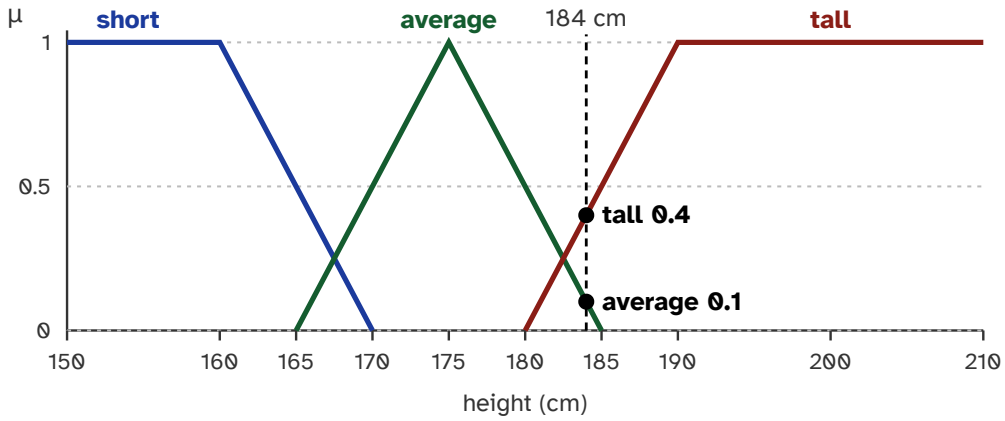
3. Pick the largest UCB1: B.
4. Why B and not A: they have the same win rate, but B has been tried fewer times, so its exploration bonus is bigger.

Answer. B (UCB1 ≈ 1.695).

P16 Read a membership and apply hedges

TOPIC 10

Problem. The graph shows the fuzzy sets short, average and tall for height in cm. For a man of 184 cm, find his membership in tall and in average, then in "very tall" and "somewhat tall".



SOLUTION, STEP BY STEP

1. Find where 184 cm is on each slope. Tall rises from 0 at 180 to 1 at 190, a slope 10 cm long.
2. 184 is 184 - 180 = 4 cm along it, so tall = 4 ÷ 10 = 0.4.
3. Average falls from 1 at 175 to 0 at 185, also 10 cm long. 184 is 185 - 184 = 1 cm before the end, so average = 1 ÷ 10 = 0.1.
4. Very = square the membership: very tall = 0.4 × 0.4 = 0.16.
5. Somewhat = square root: somewhat tall = √0.4 ≈ 0.632.
6. Check the direction: "very" is stricter, so its value is smaller (0.16 < 0.4); "somewhat" is looser, so it is bigger (0.632 > 0.4).

Answer. Tall 0.4, average 0.1, very tall 0.16, somewhat tall ≈ 0.632.

P17 Fuzzy operations: "not very tall and not very short"

TOPIC 11

Problem. A man is tall to 0.6 and short to 0.1. Find his membership in C = "not very tall and not very short", and in D = "not very very tall and not very very short".

SOLUTION, STEP BY STEP

1. Work from the inside of the phrase: hedge first, then NOT, then AND.
2. The rules: very = square (power 2), very very = power 4, NOT = 1 minus, AND = the smaller value (min).
3. Set C:

Part	Work	Value
very tall	0.6 × 0.6	0.36
not very tall	1 - 0.36	0.64
very short	0.1 × 0.1	0.01
not very short	1 - 0.01	0.99
C = AND	min(0.64, 0.99)	0.64

4. Set D:

Part	Work	Value
very very tall	0.6^4	0.1296
not very very tall	1 - 0.1296	0.8704
very very short	0.1^4	0.0001
not very very short	1 - 0.0001	0.9999
D = AND	min(0.8704, 0.9999)	0.8704

5. D is bigger than C, because "very very tall" is a stricter set, so it is easier to be outside it.

Answer. C = 0.64. D = 0.8704.

P18 Fuzzy inference: all four steps with the centroid

TOPIC 12

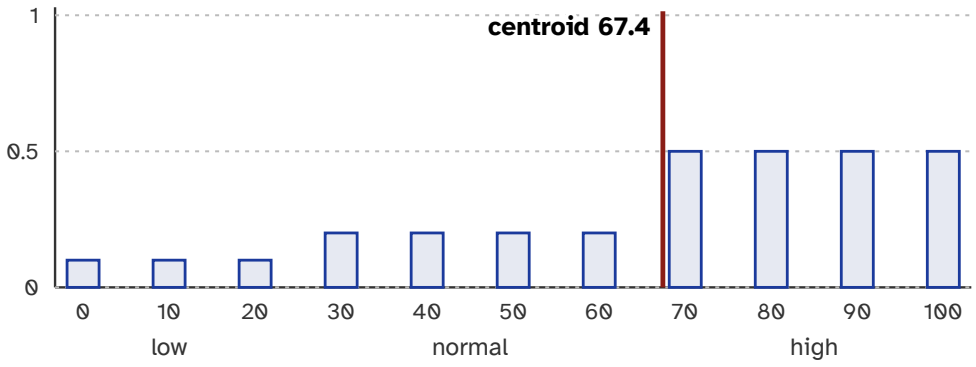
Problem. Project risk. Rule 1: IF funding is adequate (A3) OR staffing is small (B1) THEN risk is low (C1). Rule 2: IF funding is marginal (A2) AND staffing is large (B2) THEN risk is normal (C2). Rule 3: IF funding is inadequate (A1) THEN risk is high (C3). The inputs give A1 = 0.5, A2 = 0.2, A3 = 0, B1 = 0.1, B2 = 0.7. Low risk covers z = 0 to 20, normal 30 to 60, high 70 to 100 (sampled every 10). Find the risk.

SOLUTION, STEP BY STEP

1. Step 1, fuzzification: the question already gives the memberships (A1 0.5, A2 0.2, A3 0, B1 0.1, B2 0.7).
2. Step 2, rule evaluation. Use OR = max and AND = min, then clip that rule's output at the result:

Rule	Inputs	Operator	Strength	Clips
1	A3 = 0, B1 = 0.1	OR = max	0.1	low (C1)
2	A2 = 0.2, B2 = 0.7	AND = min	0.2	normal (C2)
3	A1 = 0.5	one input	0.5	high (C3)

3. Step 3, aggregation: put the three clipped outputs on one axis:



4. Step 4, centroid. Top line, each z times its height, added up: (0 + 10 + 20) × 0.1 = 3; (30 + 40 + 50 + 60) × 0.2 = 36; (70 + 80 + 90 + 100) × 0.5 = 170. Total 3 + 36 + 170 = 209.
5. Bottom line, all the heights added up: 0.1 × 3 + 0.2 × 4 + 0.5 × 4 = 0.3 + 0.8 + 2 = 3.1.
6. Divide: 209 ÷ 3.1 ≈ 67.4. This is the balance point of the shape (the red line).

Answer. Risk ≈ 67.4 out of 100: high risk.

P19 The big workshop: design a fuzzy system

TOPIC 13

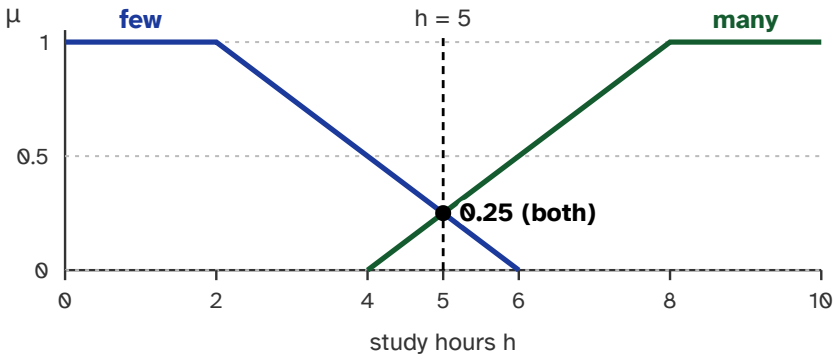
Problem. Design a fuzzy system that decides how hard to study from study hours h (0 to 10) and practice score p (0 to 100). Show the sets, membership functions, rules, the four steps for h = 5 and p = 58, and a fuzzy state machine.

SOLUTION, STEP BY STEP

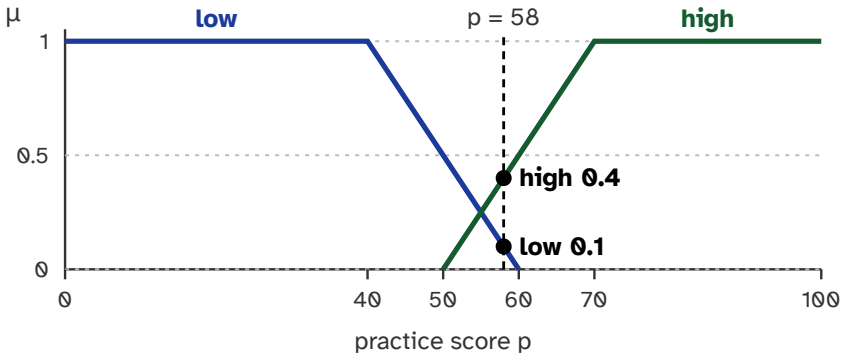
1. 1. Fuzzy sets and subsets: hours = {few, many}; score = {low, high}; urgency (0 to 100) = {rest, practice, study hard}.
2. 2. Membership functions (write the numbers):

Subset	Shape
few (hours)	1 up to 2 h, falling to 0 at 6 h
many (hours)	0 up to 4 h, rising to 1 at 8 h
low (score)	1 up to 40, falling to 0 at 60
high (score)	0 up to 50, rising to 1 at 70
rest / practice / study hard	urgency 0 to 20 / 30 to 60 / 70 to 100

3. 3. Draw the graphs:



Hours: at h = 5, few and many are both 0.25.

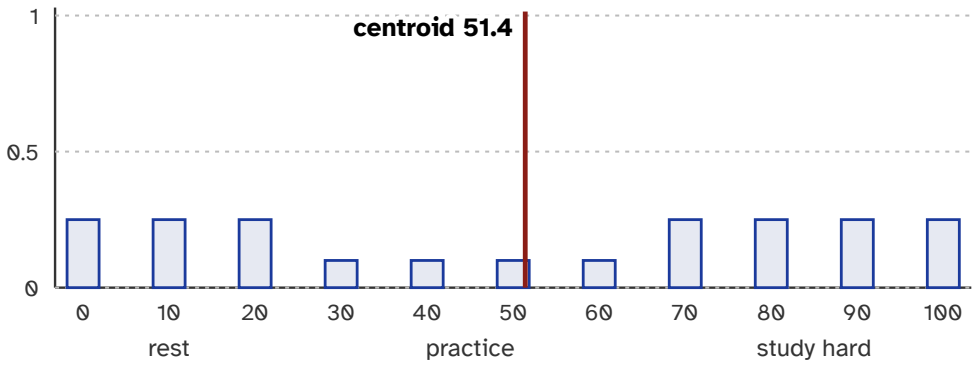


Score: at p = 58, low is 0.1 and high is 0.4.

4. 4. Rules: R1 IF hours are many AND score is high THEN rest. R2 IF hours are few OR score is low THEN study hard. R3 IF hours are many AND score is low THEN practice.
5. Fuzzification for h = 5, p = 58: few = (6 - 5) ÷ 4 = 0.25. Many = (5 - 4) ÷ 4 = 0.25. Low = (60 - 58) ÷ 20 = 0.1. High = (58 - 50) ÷ 20 = 0.4.
6. Rule evaluation (the selection process):

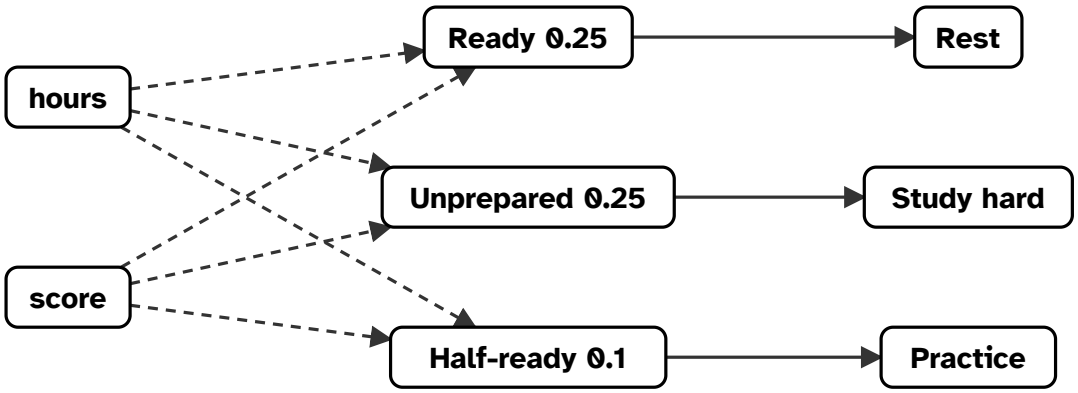
Rule	Work	Strength	Output
R1	min(many 0.25, high 0.4)	0.25	rest
R2	max(few 0.25, low 0.1)	0.25	study hard
R3	min(many 0.25, low 0.1)	0.1	practice

7. Aggregation and centroid:



8. Centroid: top = (0 + 10 + 20) × 0.25 + (30 + 40 + 50 + 60) × 0.1 + (70 + 80 + 90 + 100) × 0.25 = 7.5 + 18 + 85 = 110.5. Bottom = 0.25 × 3 + 0.1 × 4 + 0.25 × 4 = 2.15. 110.5 ÷ 2.15 ≈ 51.4.
9. Link to utility AI: each clipped rule output is a score for one action. Combining them gives one decision instead of picking a single rule.

10. 5. Fuzzy state machine, drawn from the inputs, through hidden states, to the actions (like an HMM):



Dashed arrows: the hidden part (inputs to hidden states). Each hidden state has a degree, and the machine is in all of them at once to that degree.

Answer. Urgency ≈ 51.4 out of 100: medium, so do practice tests.

P20 Naive Bayes: classify a new day

TOPIC 14

Problem. Play golf data: 14 days, 9 Yes and 5 No. The counts you need are in the table. Should we play on a Rainy, Cool, High-humidity, Windy day?

Feature value	count with Yes (of 9)	count with No (of 5)
Outlook = Rainy	2	3
Temp = Cool	3	1
Humidity = High	3	4
Windy = True	3	3

SOLUTION, STEP BY STEP

1. Priors (how common each class is): P(Yes) = 9 ÷ 14, P(No) = 5 ÷ 14.
2. Turn each count into a likelihood by dividing by the class size (9 for Yes, 5 for No):

Feature value	P(value | Yes)	P(value | No)
Rainy	2/9	3/5
Cool	3/9	1/5
High	3/9	4/5
Windy	3/9	3/5

3. Score for Yes = prior × all likelihoods: 9/14 × 2/9 × 3/9 × 3/9 × 3/9 ≈ 0.00529.
4. Score for No: 5/14 × 3/5 × 1/5 × 4/5 × 3/5 ≈ 0.02057.
5. Compare: 0.02057 is bigger, so the answer is No.
6. If the question asks for probabilities, divide each score by their sum: 0.00529 + 0.02057 = 0.02586. P(No) = 0.02057 ÷ 0.02586 ≈ 0.80, P(Yes) ≈ 0.20.

Answer. Do not play (No ≈ 0.80, Yes ≈ 0.20).

P21 Naive Bayes with a zero count (Laplace smoothing)

TOPIC 14

Problem. Same golf data (9 Yes, 5 No). New day: Overcast, Mild, High humidity, not Windy. Outlook and Temperature each have 3 possible values; Humidity and Windy have 2. Classify the day, then redo it with Laplace smoothing.

Feature value	count with Yes (of 9)	count with No (of 5)
Outlook = Overcast	4	0
Temp = Mild	4	2
Humidity = High	3	4
Windy = False	6	2

SOLUTION, STEP BY STEP

1. Without smoothing, Yes: 9/14 × 4/9 × 4/9 × 3/9 × 6/9 ≈ 0.0282.
2. Without smoothing, No: the Overcast count is 0, so P(Overcast | No) = 0/5 = 0, and the whole product is 0. One zero wipes out all the other evidence: this is the zero-probability problem.
3. Laplace smoothing: add 1 to every count, and add the number of possible values to the bottom:

Feature value	Yes	No
Overcast (3 values)	(4+1)/(9+3) = 5/12	(0+1)/(5+3) = 1/8
Mild (3 values)	(4+1)/(9+3) = 5/12	(2+1)/(5+3) = 3/8
High (2 values)	(3+1)/(9+2) = 4/11	(4+1)/(5+2) = 5/7
not Windy (2 values)	(6+1)/(9+2) = 7/11	(2+1)/(5+2) = 3/7

4. Yes with smoothing: 9/14 × 5/12 × 5/12 × 4/11 × 7/11 ≈ 0.0258.
5. No with smoothing: 5/14 × 1/8 × 3/8 × 5/7 × 3/7 ≈ 0.00512.
6. Compare: 0.0258 is bigger, so Yes. As a probability: 0.0258 ÷ (0.0258 + 0.00512) ≈ 0.834.

Answer. Play golf. Without smoothing the No score is 0; with Laplace, Yes ≈ 0.834.

P22 Gaussian likelihood for a number

TOPIC 14

Problem. For play = Yes, humidity has mean μ = 79.1 and standard deviation σ = 10.2. Find P(humidity = 74 | Yes) with the normal (Gaussian) formula.

SOLUTION, STEP BY STEP

1. Formula: $1 \div (\sqrt{(2\pi)} \times \sigma) \times e^{-(x - \mu)^2 \div (2\sigma^2)}$. Here x = 74.
2. Do it in small pieces:

Piece	Work	Value
$x - \mu$	74 - 79.1	-5.1
$(x - \mu)^2$	-5.1 × -5.1	26.01
$2\sigma^2$	2 × 10.2 × 10.2	208.08
exponent	-26.01 ÷ 208.08	≈ -0.125
e^{exponent}	$e^{-0.125}$ (calculator e^x)	≈ 0.8825
front part	1 ÷ (2.5066 × 10.2) = 1 ÷ 25.568	≈ 0.03911

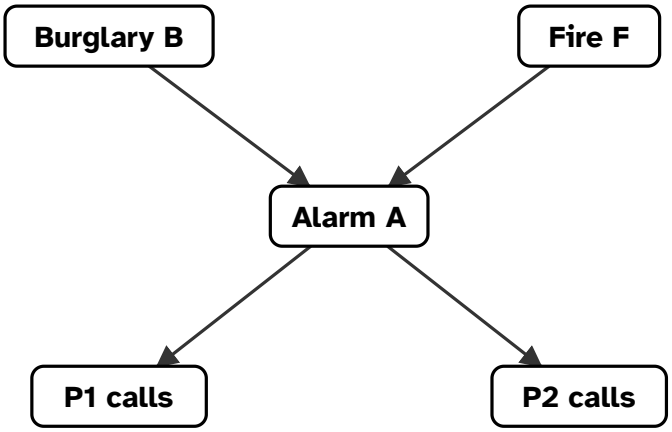
3. Multiply the front part by e to the exponent: 0.03911 × 0.8825 ≈ 0.0345.

Answer. ≈ 0.0345 (the slide shows 0.0344 from rounding).

P23 Bayesian network: probability of a full situation

TOPIC 15

Problem. Use the network and its tables below. Find the probability that both neighbours call and the alarm rings, but there is no burglary and no fire: P(P1, P2, A, no B, no F).



	P(true)
Burglary	0.001
Fire	0.002

B	F	P(A = true)
T	T	0.95
T	F	0.94
F	T	0.29
F	F	0.001

A	P(P1 = true)	P(P2 = true)
T	0.95	0.80
F	0.05	0.01

SOLUTION, STEP BY STEP

1. Teacher's method: look at the arrow heads. Burglary and Fire have no arrows coming in, so write P(no B) and P(no F).
2. Alarm has arrows from Burglary and Fire, so write P(A | no B, no F).
3. P1 and P2 each have one arrow, from Alarm, so write P(P1 | A) and P(P2 | A).
4. Write the whole statement before any numbers: P(P1 | A) × P(P2 | A) × P(A | no B, no F) × P(no B) × P(no F).
5. Look up each number:

Piece	Where it comes from	Value
P(P1 | A)	P1 table, row A = T	0.95
P(P2 | A)	P2 column, row A = T	0.80
P(A | no B, no F)	Alarm table, row F, F	0.001
P(no B)	1 - 0.001	0.999
P(no F)	1 - 0.002	0.998

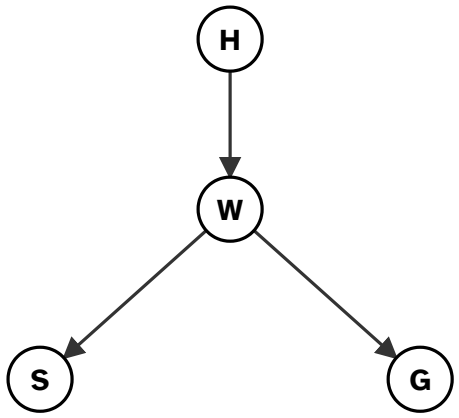
6. Multiply step by step: 0.95 × 0.80 = 0.76. Then 0.76 × 0.001 = 0.00076. Then 0.00076 × 0.999 × 0.998 ≈ 0.000758.

Answer. ≈ 0.000758. A false alarm where both neighbours call is very unlikely.

P24 Bayesian network with a hidden node

TOPIC 15

Problem. Use the network and tables below. Find P(+w | +s, -g): the probability that W is true when S is true and G is false.



	P(true)
H	0.1

H	P(+w)
+h	0.667
-h	0.25

W	P(+s)	P(-g)
+w	0.4	0.75
-w	0.2	0.5

SOLUTION, STEP BY STEP

1. H is not mentioned in the question, so it is hidden. Work out each case of H and add them up.
2. Fill in the missing values with 1 minus: P(-h) = 0.9. P(-w | +h) = 1 - 0.667 = 0.333. P(-w | -h) = 1 - 0.25 = 0.75.
3. Each case is a product down the network: P(h) × P(w | h) × P(+s | w) × P(-g | w).
4. Work out all four cases:

Case	Product	Value
+h, +w	0.1 × 0.667 × 0.4 × 0.75	0.0200
-h, +w	0.9 × 0.25 × 0.4 × 0.75	0.0675
+h, -w	0.1 × 0.333 × 0.2 × 0.5	0.0033
-h, -w	0.9 × 0.75 × 0.2 × 0.5	0.0675

5. Top of the fraction = the +w cases: 0.0200 + 0.0675 = 0.0875.
6. Bottom of the fraction = all four cases: 0.0875 + (0.0033 + 0.0675 = 0.0708) = 0.1583.
7. Divide: 0.0875 ÷ 0.1583 ≈ 0.553.

Answer. ≈ 0.553.

P25 Dynamic time warping table

TOPIC 16

Problem. Find the DTW distance between X = (1, 2, 3) and Y = (2, 2, 4), and the warping path.

SOLUTION, STEP BY STEP

1. Step 1, the distance table C: each cell is |x - y| for that row and column:

X \ Y	2	2	4
1	1	1	3
2	0	0	2
3	1	1	1

Table C. Rows are X, columns are Y.

2. Step 2, the cost table D. The first row is a running total of C's first row: 1, 1 + 1 = 2, 2 + 3 = 5. The first column is a running total of C's first column: 1, 1 + 0 = 1, 1 + 1 = 2.
3. Every other cell: D = C + the smallest of the three cells above, to the left, and diagonally up-left.
4. D(2, 2) = 0 + min(2, 1, 1) = 1. D(2, 3) = 2 + min(5, 1, 2) = 3. D(3, 2) = 1 + min(1, 2, 1) = 2. D(3, 3) = 1 + min(3, 2, 1) = 2.
5. The full table D, with the path marked:

X \ Y	2	2	4
1	**1**	2	5
2	1	**1**	3
3	2	2	**2**

Table D. Marked cells are the warping path.

6. Step 3, the distance is the bottom-right cell: D(3, 3) = 2.
7. Step 4, the path: from the bottom-right cell, move to the smallest of the three neighbours each time: (3, 3) to (2, 2) to (1, 1). All moves are diagonal, which means each point is matched one to one.

Answer. DTW distance 2. Path (1,1), (2,2), (3,3).